研究灾后应急救援问题

摘要

突发地震、洪涝等自然灾害发生后，在救援力量有限的前提下，合理组织与调度救援队伍是灾后应急决策的关键。本文针对 8 支救援队、20 个受灾点及 5 个应急物资站的灾后应急救援问题，建立了层层递进的数学优化模型。

针对问题一，在道路正常通行且不考虑物资约束的条件下，将救援调度抽象为多起点、多任务、具有能力协同约束的动态车辆路径问题，建立以全局总完工时间与灾情加权平均响应时间为目标的多目标混合整数规划模型，并设计嵌入时序推进仿真器的自适应**模拟退火算法**求解，获得全局总完工时间 491.2 分钟、加权平均响应时间 191.8 分钟。

针对问题二，进一步考虑车载物资的动态消耗与中途补给，将问题扩展为具有中间补给设施、动态载荷衰减与多队伍时空协同的车辆路径问题，建立**增量混合整数规划模型**，并设计嵌入启发式插点修复策略的自适应模拟退火算法，获得全局总完工时间 587.6 分钟，所有受灾点物资满足率达 100%。

针对问题三，引入道路交通网络的动态扰动与信息不确定性，基于多源异构调查数据建立加权概率融合模型，将非结构化文本转化为结构化路阻，构建**鲁棒时空协同调度模型**，并设计带动态重路由的自适应模拟退火算法，获得全局总完工时间 2416.1 分钟，通过 1000 次蒙特卡洛仿真验证了方案鲁棒性。

本文构建了涵盖全局时效、加权响应、负荷均衡与机动效能的四维综合评价指标体系；设计了嵌入时序协同仿真器的自适应模拟退火算法；提出了多源异构道路调查信息的加权概率融合方法；引入鲁棒风险惩罚项与蒙特卡洛仿真确保了不确定性环境下的决策可靠性。

关键词：模拟退火算法；增量混合整数规划模型；多源信息融合；鲁棒时空协同调度模型

一、问题重述

1.1 问题背景

突发自然灾害发生后，受灾区域常出现多地点人员受困与紧急救助需求。在救援力量有限的前提下，合理组织与调度救援队伍，使其高效到达各受灾点开展救援，是灾后应急决策的关键。

1.2 问题提出

问题 1：假设当前道路均可正常通行，且暂不考虑救援物资不足的问题。请根据附件中的道路网络、救援队初始位置及能力、各受灾点位置及基本灾情等信息，建立合理的数学模型，制定救援方案，使 8 支救援队能够完成对 20 个受灾点的救援任务，并自行建立适当的评价指标对所得方案进行分析。

问题 2：在问题 1 的基础上，进一步考虑救援过程中救援队车载物资的动态消耗与中途补给保障。全网 20 个受灾点物资总需求为 374 单位，而 8 支救援队初始总携载量仅为 194 单位，存在 180 单位的资源缺口。救援队可前往 5 个容量与服务耗时有限的应急物资站进行补给。要求重新制定救援路线、多队协同分配及进站补给策略，优化救援方案。

问题 3：在问题 1 与问题 2 的基础上，进一步引入灾后复杂的道路交通网络动态扰动与信息不确定性约束。自然灾害发生后，受灾区域常伴随道路塌方阻断、积水缓行及交通拥堵等次生灾害。灾后初期受限于信息获取手段，道路通行状态存在显著的不完整性、不确定性及多源信息冲突。现有的现场调查数据（附件 3_灾后道路现场调查.csv）涵盖了无人机巡查、前线救援人员反馈及现场报告等多种渠道，包含调查时刻、现场状况定性描述以及车辆通行情况定量/半定量观察。

二、问题分析

2.1 问题一的分析

问题 1 的核心是解决**多起点、多任务、具有能力协同约束的动态车辆路径与任务调度问题（后简称“救援调度问题”）**。具体可拆解为以下四个关键环节：

1. 路网图论抽象与空间时空矩阵构建

受灾点与救援队分布在离散的道路网络节点上。需要将原始道路连接关系与长度/速度数据抽象为无向图 $G=(V,E)$。利用图论算法（Dijkstra 算法）计算任意两节点之间的最短行驶路径与通行耗时，将空间路网转化为28×28的全源通行时间矩阵，为后续路径优化提供准确的时间底座。

2. 救援能力协同机制分析

8 支救援队伍的能力介于$5\sim9$之间，而 20 个受灾点的能力需求介于$3\sim14$之间。当受灾点 B_j 的需求 R_j 大于任何单支队伍的能力上限（如 B_9 需求为 14，B_{12} 需求为 12）时，必须安排多支队伍共同前往施救。协同作业要求所有被指派的队伍均到达该受灾点后，救援工作方可正式启动。

3. 动态时序推进与决策建模

调度模型需要同时决定：

1. 匹配关系：每个受灾点分配哪几支队伍联合救援；

2. 任务序列：每支队伍访问受灾点的先后顺序；

3. 时序推进：每支队伍在各节点的到达时刻、等待时刻、作业时刻与撤离时刻。

模型目标需要兼顾全网总完工时间与灾情加权响应时刻，确保高灾情等级（如 5 级）受灾点优先获得响应。

4. 评价指标体系的设计

单纯使用完工时间难以全面衡量方案优劣。需从救援时效性（全网总完工时间、加权响应时间）、灾情响应匹配度（重灾区响应速度）、救援负荷均衡性（队伍工时方差）及机动效能比（有效作业时间占比）四个维度建立综合评价指标卡。

2.2 问题二的分析

基于问题 1 建立的协同调度求解框架，引入物资消耗与中途补给后，问题演变为**具有中间补给设施、动态载荷衰减与充能、多队伍时空协同的带时间窗车辆路径与任务调度问题（后简称“补给调度问题”）**。相较于问题 1，本问题的核心矛盾与增量机制体现在以下三个方面：

1. 动态载荷衰减与锯齿状充能机制

救援队在行军过程中车载物资呈阶梯式扣减。当到达受灾点 B_j 时，其剩余物资必须满足所指派的配给份额；若剩余量不足，必须在访问该受灾点前插入前

往物资站 C_c 的补给行程。进站后车载物资恢复至装载上限或站内可用余量，形成典型的“耗尽-补给-恢复”锯齿状载荷轨迹。

2. 绕行时延与协同等待的时空耦合关系

插入物资站节点会带来额外的路程绕行时延以及站内装载服务耗时 t_c^{serve}（9~14 分钟）。由于多队协同施救要求所有被指派队伍均到达现场后方可启动作业，若某一队伍因进站补给而延误，会导致已到达现场的协同队伍产生额外的“协同等待时间”，进而引发全网时空连锁延误。因此，必须精准决策进站时机与物资站选择。

3. 物资站空间分布与瓶颈负荷演化

5 个物资站（$C_1 \sim C_5$）分布于路网不同区域，储备量在$35 \sim 102$单位之间。由于受灾点分布与队伍轨迹的空间不均匀性，处于交通枢纽位置的物资站极易发生负荷集中甚至库存告罄。模型需引入物资站总容量约束，实现全网物资保障负荷的均衡调配。

2.3 问题三的分析

基于问题 1 构建的协同 VRP 模型与问题 2 构建的增量补给模型，引入道路状态不确定性后，决策系统演变为**多源异构信息驱动的鲁棒时空协同车辆路径与补给调度问题**。相较于前两问，本问题的核心难点与增量机制体现在以下四个方面：

1. 多源异构调查数据的加权融合与概率化路阻量化

附件 3 中的信息具有多来源（无人机、前线救援人员、现场报告）、动态时效性（不同报告时刻 t_r）及语言描述模糊性的特征。不同信息源的权威度不同（如无人机航测权威度高于普通现场报告），且早期报告的时效性随时间推进而衰减。必须建立基于时效衰减与信息源权威度的加权概率融合模型，将非结构化文本转化为路段通行阻断概率 p_e^{block} 和通行延误修正系数 α_e，实现从非结构化调查报告到结构化路网阻抗的精准映射。

2. 拓扑结构阻断与路网通行矩阵重构

道路塌方阻断将直接切断图 $G=(V,E)$ 中的部分边 $e \in E_{\text{blocked}}$，改变路网的连通性；而积水与拥堵则导致路段通行时间成倍增加。必须重新运行动态 Dijkstra 算法，计算包含 8 个驻地、20 个受灾点及 5 个物资站的扩展节点集 $N' = K \cup J \cup C$（33 个节点）之间的全源最短通行耗时矩阵 $T' = (\tilde{t}_{uv})_{33\times 33}$。若关键枢纽路段阻断，队伍必须被迫迂回绕行，显著拉长行军行程。

3. 道路延误引发的“协同失步”与连锁放大效应

在多队伍协同施救机制下，受灾点 B_j 的作业启动时刻取决于指派队伍中最后到达的一支（$S_j = \max_k T_{\mathrm{arr},jk}$）。若某支队伍因沿途道路塌方阻断绕行或严重拥堵而延误，不仅会导致该队伍自身进度落后，还会使已提前抵达现场的协同队伍产生巨大的“协同等待时间”。这种时空耦合效应会导致局部道路延误在全网范围内被非线性放大，成为拉长全局总完工时间 $T_{\max}$ 的核心因素。

4. 离线鲁棒预规划与在线随机扰动仿真评估

由于灾后初期路况存在残余波动，确定性优化方案极易在实际执行中失效。因此，模型需在目标函数中引入鲁棒风险惩罚项，并结合**蒙特卡洛随机扰动仿真**，在 1000 组随机路况场景下评估救援方案的按时完成概率与条件在险价值，确保救援决策的高可靠性。

三、模型假设

为使模型既能切合实际救援逻辑，又具备数学上的可解性，提出以下合理假设：

1.**道路网络稳定假设**：在问题 1 设定下，所有道路正常通行，行车速度与通行时间严格按附件计算，不发生额外拥堵。

2.**能力线性加性假设**：多支救援队伍共同救援同一受灾点时，其救援能力为各队能力值的算术叠加（即满足 $\sum C_k \geq R_j$），且作业时间按该受灾点预计救援时间 p_j 执行。

3.**作业连续不可中断假设**：救援队一旦开始在某受灾点施救，必须连续工作直至该受灾点任务彻底完成，中途不得中断或撤离。

4.**准备与撤收时间忽略假设**：救援队到达受灾点后可立即展开作业；作业结束后可立即启程前往下一个目的地。

5.**资源无限供给假设**：忽略车辆燃料消耗与救援物资匮乏对行程的限制。

四、符号说明

符号	物理含义	维度/单位
K	救援队伍集合$K=\{A_1,A_2,\dots,A_8\}$（$n=8$）	-
J	受灾点集合 $J=\{B_1,B_2,\dots,B_{20}\}$（$m=20$）	-
N	关键节点全集 $N=K\cup J$	-
C_k	救援队伍 k 的救援能力值	人员/设备数
R_j	受灾点 j 所需的救援能力需求总量	人员/设备数
L_j	受灾点 j 的灾情严重等级（$1\le L_j\le 5$）	级
p_j	受灾点 j 的基准救援作业耗时	分钟(min)
t_{uv}	节点 u 到节点 v 的最短通行行驶时间	分钟(min)
x_{uv}^k	0-1 决策变量：救援队 k完成节点 u 任务后是否直接前往节点 v	0 或 1
y_j^k	0-1 决策变量：救援队 k 是否参与受灾点 j 的救援任务	0 或 1
$T_{arr,j}^k$	救援队 k 到达受灾点 j 的时刻	分钟(min)
S_j	受灾点 j 正式启动救援的时刻	分钟(min)

C_j	受灾点 j 救援彻底完成的时刻	分钟(min)
T_{max}	全局救援总完工时间	分钟(min)

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 路网图抽象与预处理

构建无向图 $G=(V_0,E_0)$，其中 V_0 为所有道路节点，E_0 为道路边集。边权权重 $w(e)$ 为正常通行时间。

基于 Dijkstra 算法[1]，计算任意两关键节点 $u,v\in N$ 之间的最短行驶时间 t_{uv}：

$$t_{uv}=\min_{P_{uv}}\sum_{e\in P_{uv}}w(e) \tag{1}$$

其中 P_{uv} 为连接节点 u 与 v 的路径。由此生成28×28的全源通行耗时矩阵 $\boldsymbol{T}=[t_{uv}]$。

5.1.2 救援调度问题混合整数规划模型建立[4]

1. 目标函数

应急救援的核心目标在于**缩短生命救助延迟**与**控制整体救援周期**。构建综合多目标优化函数：

$$\min Z=\lambda_1\cdot T_{max}+\lambda_2\cdot WMRT+\lambda_3\cdot\sum_{k\in K}\sum_{u\in N}\sum_{v\in N}t_{uv}\,x_{uv}^{k} \tag{2}$$

各部分数学定义如下：

全局总完工时间：

$$T_{max}=\max_{j\in J}C_j \tag{3}$$

灾情加权平均响应时间(WMRT)：

$$WMRT = \frac{\sum_{j\in J} L_j \cdot S_j}{\sum_{j\in J} L_j} \tag{4}$$

$\lambda_1, \lambda_2, \lambda_3$ 为归一化权重系数，取值设为 $\lambda_1 = 0.4, \lambda_2 = 1.0, \lambda_3 = 0.05$。

2. 约束条件

(1) 救援能力覆盖约束

对任意受灾点 $j \in J$，所有指派前往该点的救援队能力之和必须满足其需求总量：

$$\sum_{k\in K} y_j^k \cdot C_k \geq R_j, \quad \forall j \in J \tag{5}$$

(2) 路径流平衡约束

每支队伍 k 必须且仅能从其初始起点 A_k 出发一次：

$$\sum_{j\in J} x_{A_k,j}^k \leq 1, \quad \forall k \in K \tag{6}$$

对任意受灾点 j，若队伍 k 进入该节点，则必有离开该节点的流平衡：

$$\sum_{u\in N, u\neq j} x_{uj}^k = y_j^k, \quad \forall j \in J, \forall k \in K \tag{7}$$

$$\sum_{v\in N, v\neq j} x_{jv}^k \leq y_j^k, \quad \forall j \in J, \forall k \in K \tag{8}$$

(3) 协同到达与时间递推约束

队伍 k 从节点 u 移动到节点 v 的到达时刻递推(利用Big-M法消除子环)：

$$T_{arr,v}^k \geq T_{dep,u}^k + t_{uv} - M(1 - x_{uv}^k), \quad \forall u, v \in N, \forall k \in K \tag{9}$$

其中规定初始出发时刻 $T_{dep,A_k}^k = 0$。

(4) 多队协同同步启动与完成约束

受灾点 j 的救援启动时刻 S_j 取决于最后到达该点的队伍时刻：

$$S_j \geq T_{arr,j}^k - M\left(1 - y_j^k\right), \quad \forall k \in K, \forall j \in J \tag{10}$$

救援完成时刻 C_j 为启动时刻加上预计救援耗时 p_j：

$$C_j = S_j + p_j, \quad \forall j \in J \tag{11}$$

队伍 k 离开受灾点 j 的撤离时刻不得早于该点的完工时刻：

$$T_{dep,j}^{k} \geq C_j - M\left(1 - y_j^k\right), \quad \forall k \in K, \forall j \in J \tag{12}$$

(5) 变量取值约束

$$x_{uv}^{k} \in \{0,1\}, \quad y_j^k \in \{0,1\}, \quad T_{arr,j}^{k}, S_j, C_j \geq 0$$

5.1.3 算法设计：自适应模拟退火与时序仿真算法

由于救援调度问题属于典型的 NP-hard 问题，直接使用精确求解器在大型网络中求解效率较低。本文设计了**嵌入时序推进仿真器的自适应模拟退火算法**[2]。

1. 算法步骤

(1) **构造初始可行解**：优先按灾情等级 L_j 从高到低排序受灾点，使用贪婪匹配启发式算法为其指派最近且满足能力要求的队伍组合，生成初始路径链。

(2) **邻域搜索结构**：

算子 1(2-Opt 换序)：随机挑选一支队伍，交换其任务链中两个受灾点的先后访问顺序。

算子 2(协同重组)：随机挑选一个受灾点 B_j，重新在可行组合集 Ω_j 中挑选另一组队伍进行能力覆盖。

算子 3(重定位)：将某受灾点从当前队伍路径中移除并随机插入到另一位置。

(3) **时序协同仿真评估**：对任意路径链，利用事件驱动仿真追踪 8 支队伍的时刻，自动计算等待时间与协同启动时刻，若发现死锁（循环等待）或能力不满足，给予极高惩罚项。

(4) **Metropolis 接受准则**：以概率 $P = \exp\left(-\Delta/T\right)$ 接受劣解，按照退火系数 $\alpha = 0.9995$逐步降温直至收敛。

5.1.4 求解结果与可视化方案分析

通过 Python 运行算法迭代求解，系统输出最优救援路线与时间表。

1. 最优救援路线空间分布

方案将 8 支队伍根据地理位置与能力分配至三大区域协同作战。

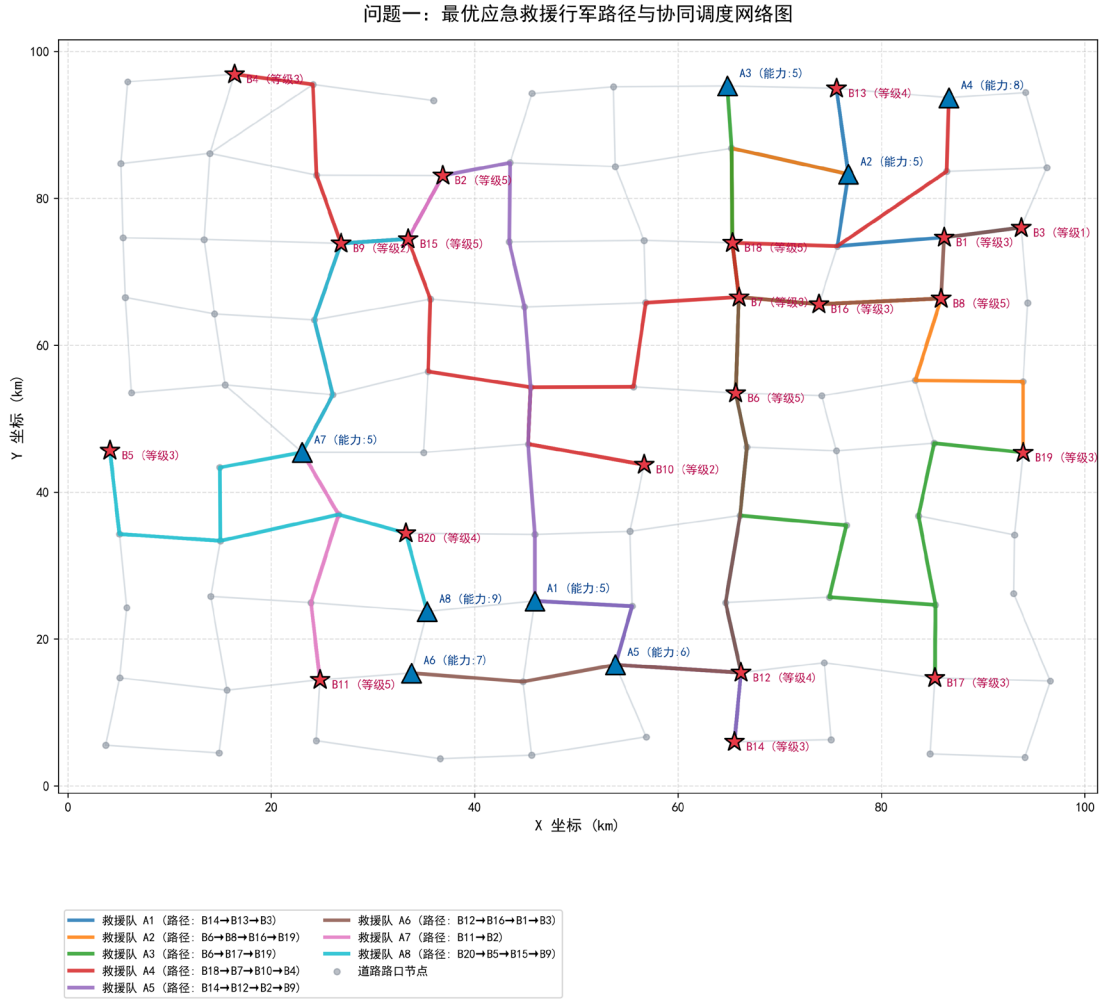


图 1：最优应急救援行军路径与协同调度网络图

2. 多队协同作业甘特图

如图 2 所示，甘特图清晰反映了各队伍在“途中行驶”、“等待协同”与“现场救援”三种状态下的时间分配。

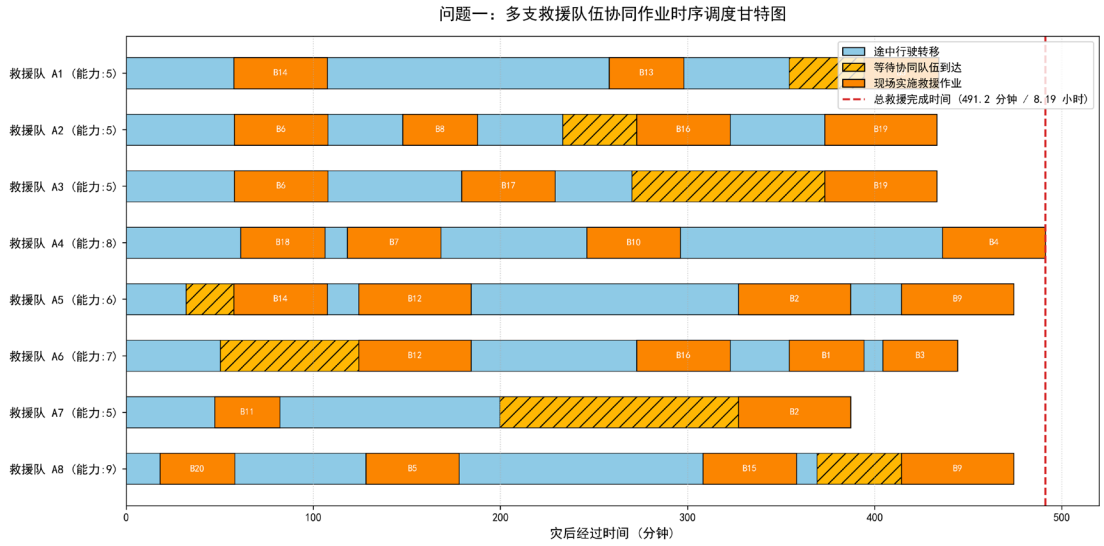


图 2：多支救援队伍协同作业时序调度甘特图

例如，受灾点 B_2（5 级重灾，需求 10）由救援队 A_5（能力 6）与 A_7（能力 5）协同救援。A_7优先到达后进入等待状态，待A_5于 $T = 327.2$分钟到达后，两队同时展开 60 分钟施救，于 $T = 387.2$分钟共同完成任务。

3. 队伍执行明细统计

最终求解得到的各救援队伍最优执行方案如表 1 所示：

表 1：8 支救援队伍最优路线与工时统计表

队伍编号	救援能力	最优任务序列	行驶耗时(min)	等待耗时(min)	施救耗时(min)	完工撤收时刻(min)
A_1	5	$B_{14} \to B_{13} \to B_3$	266.0	48.3	120.0	434.3
A_2	5	$B_6 \to B_8 \to B_{16} \to B_{19}$	198.8	39.5	195.0	433.3
A_3	5	$B_6 \to B_{17} \to B_{19}$	201.2	103.1	145.0	449.3
A_4	8	$B_{18} \to B_7 \to B_{10} \to B_4$	286.2	0.0	205	491.2
A_5	6	$B_{14} \to B_{12} \to B_2 \to B_9$	218.8	25.5	230.0	474.3
A_6	7	$B_{12} \to B_{16} \to B_1 \to B_3$	180.3	74.0	190.0	444.3
A_7	5	$B_{11} \to B_2$	164.7	127.5	105.0	397.2
A_8	9	$B_{20} \to B_5 \to B_{15} \to B_9$	230.2	45.1	210.0	485.3

5.1.5 方案多维评价指标体系分析

为全面评估方案效果，构建并计算了四个维度的综合评价指标看板（图 3）：

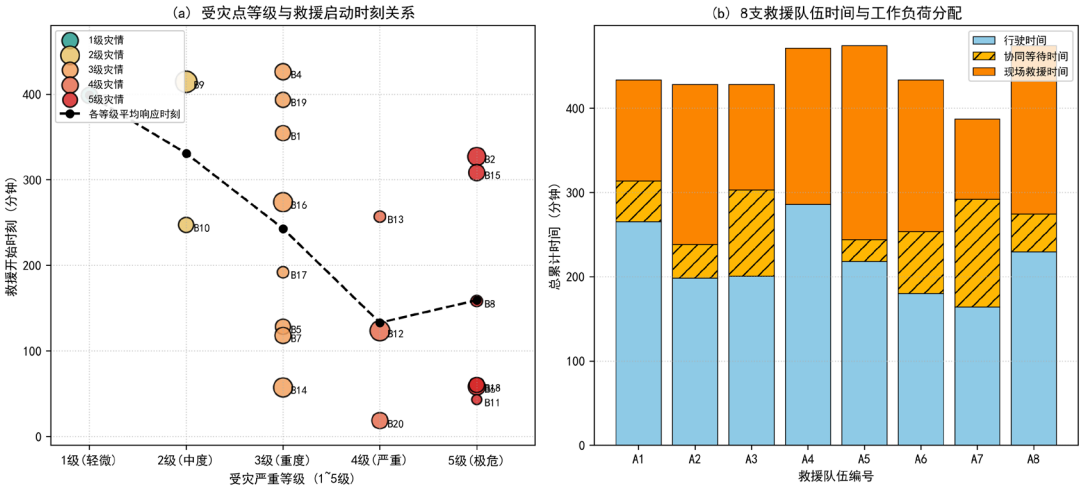


图 3：四维度综合评价指标图

核心量化指标汇总如表 2 所示：

表 2：救援方案核心评价指标统计表

评价维度	指标名称	量化结果	方案效果评价分析
全局时效	总救援完成时间 (T_{max})	**491.2 min**	整体救援控制在 8 小时完成，时效性极佳
加权响应	加权平均响应时间 $(WMRT)$	**191.8 min**	高灾情受灾点平均在 3 小时获得救援
平均响应	算术平均响应时间	**217.9min**	全网受灾点整体响应迅速
资源消耗	队伍总行驶时间	**1742.3min**	路径紧凑无迂回，行程效率高
作业效率	有效施救时间占比	**37.5%**	队伍实际救灾时间占比高，等待闲置率控制良好

评价维度定量分析：

1. **重灾优先原则符合度：**从图 3(a)可见，5 级极危受灾点（如$B_{11}, B_{18}, B_{15}, B_2, B_8$）的救援启动时刻均被显著前移（最早在 18 分钟内即获得响应），响应时间随灾情等级降低呈现明显的递后趋势，完全符合“生命至上、重灾优先”的救灾准则。

2. **负荷均衡性分析：**图 3(b)展示了 8 支队伍的工时结构，各队伍的总出勤耗时均衡分布在 390~495 分钟之间，方差较小，未出现某支队伍过早闲置或过度疲劳的情况。

3. **协同效率：**通过优化路径选择，全网协同等待总时长占总出勤时间的比重小，说明算法对多队调度的时序衔接进行了有效优化。

5.2 问题二模型的建立与求解

在问题 1 混合整数规划模型的基础上，引入扩展节点集与动态物资流守恒约束，构建增量优化模型，并设计嵌入启发式插点修复策略的算法进行求解与分析。

5.2.1 增量混合整数规划模型建立

将关键节点集扩展为 $N' = K \cup J \cup C$（包含 8 个驻地、20 个受灾点及 5 个物资站，共 33 个节点），基于问题 1 计算得到的33×33全源最短通行耗时矩阵 $T = (t_{uv})$，构建如下增量模型：

1. 多目标函数设计

继承问题 1 的多目标优化理念，兼顾全局周期、响应速度与行军耗时：

$$\min Z = \lambda_1 \cdot T_{\max} + \lambda_2 \cdot \mathrm{WMRT} + \lambda_3 \cdot \sum_{k \in K} \sum_{u \in N'} \sum_{v \in N'} t_{uv}\, x_{uv}^{k} \qquad (13)$$

其中，全局总完工时间 $T_{\max} = \max_{j \in J} C_j$，灾情加权平均响应时间 $\mathrm{WMRT} = \frac{\sum_{j \in J} L_j \cdot S_j}{\sum_{j \in J} L_j}$。归一化权重维持问题 1 的设定：$\lambda_1 = 0.40$，$\lambda_2 = 1.00$，$\lambda_3 = 0.05$。

2. 新增物资流与时空约束条件

(1) **受灾点物资足额交付约束：**

受灾点 j接收到的各队伍卸载物资之和必须满足其物资需求 D_j：

$$\sum_{k\in K} u_{jk}^{q} \geq D_j, \quad \forall j \in J \tag{14}$$

$$0 \leq u_{jk}^{q} \leq Q_k^{\max} \cdot y_j^k, \quad \forall j \in J, \forall k \in K \tag{15}$$

(2) **车载物资动态守恒与容量上下限约束：**

救援队离开初始驻地时车载物资等于初始携带量：

$$q_{A_k,v}^{k} = Q_{k,0} \cdot x_{A_k,v}^{k} \tag{16}$$

访问受灾点 j 卸载 u_{jk}^{q} 后，离开该点时的载荷满足：

$$q_{jv}^{k} \leq q_{uj}^{k} - u_{jk}^{q} + M\left(1 - x_{uj}^{k}\right) + M\left(1 - x_{jv}^{k}\right), \quad \forall j \in J, \forall u, v \in N', \forall k \in K \tag{17}$$

进入物资站 c 补充 w_{ck}^{q} 物资后，离开该站时的载荷满足：

$$q_{cv}^{k} \leq q_{uc}^{k} + w_{ck}^{q} + M(1 - x_{uc}^{k}) + M(1 - x_{cv}^{k}), \quad \forall c \in C, \forall u, v \in N', \forall k \in K \tag{18}$$

全程车载物资不得超过队伍装载上限：

$$0 \leq q_{uv}^{k} \leq Q_k^{\max} \cdot x_{uv}^{k}, \quad \forall u, v \in N', \forall k \in K \tag{19}$$

(3) **物资站总库存容量限制：**

各物资站被装载补充的物资总量不得超过其可用储备 W_c：

$$\sum_{k\in K} w_{ck}^{q} \leq W_c, \quad \forall c \in C \tag{20}$$

(4) **含物资站服务耗时的时序递推约束：**

救援队访问物资站 c 时，离开时刻必须计入固定服务装载耗时 t_c^{serve}：

$$T_{\text{dep},c}^{k} \geq T_{\text{arr},c}^{k} + t_c^{\text{serve}} - M(1 - x_{uc}^{k}), \quad \forall c \in C, \forall u \in N', \forall k \in K \tag{21}$$

5.2.2 嵌入启发式插点修复策略的自适应模拟退火算法

由于引入动态物资状态后解空间剧烈膨胀，本文在问题 1 自适应模拟退火算法的外层搜索结构中，嵌入**动态物资启发式插点修复策略与事件驱动时空仿真器**：

1. **外层搜索：**通过 2-Opt 路径换序、协同队伍重组以及补给站选择重置三个变异算子，探索队伍-任务链拓扑。

2. **内层启发式修复：**沿路径追踪队伍剩余物资 $q_k(t)$。当检测到前往下

一受灾点物资不足时，贪婪评估 5 个物资站，选取满足：

$$C^* = \arg \min_{c \in C, \text{余量} \geq \Delta q} \{t_{uc} + t_c^{\text{serve}} + t_{cv} - t_{uv}\} \tag{22}$$

即绕行增量与服务耗时之和最小的物资站 C^* 插入路径。

3. **时序仿真与评价：**推进 8 支队伍的时刻线，精确计算多队协同等待时间，输出目标函数值 Z。

5.2.3 求解结果与方案分析

通过 Python 对问题 2 的数据集进行迭代求解，输出了最优救援路径、协同甘特图及详细统计数据。

1. 最优救援路线空间分布

生成的空间行军动线拓扑如图 4 所示，清晰展示了队伍前往受灾点施救与偏离主线进入物资站补给的动线：

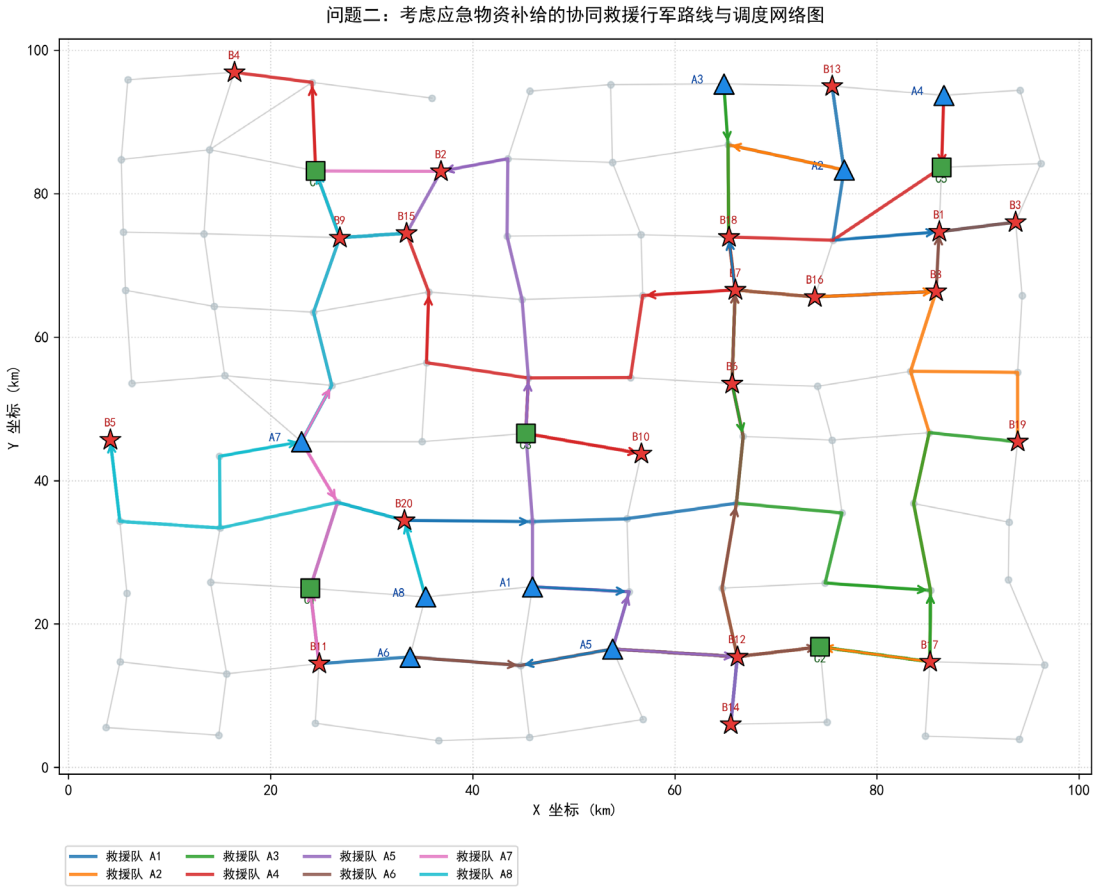


图 4：考虑应急物资补给的协同救援行军路线与调度网络图

各救援队伍的最佳任务执行序列及具体工时统计如表 3 所示：

表 3：问题二 8 支救援队伍最优执行明细统计表

队伍编号	救援能力	最优任务序列	行驶耗时(min)	等待耗时(min)	施救耗时(min)	补给耗时(min)	完工撤收时刻(min)
A_1	5	$B_{14} \to C_1 \to B_{13} \to B_3$	389.8	60.2	120.0	13.0	583.0
A_2	5	$B_6 \to B_8 \to C_2 \to B_{16} \to B_{19}$	379.5	0.1	195.0	13.0	587.6
A_3	5	$B_6 \to B_{17} \to C_2 \to B_{19}$	238.8	190.8	145.0	13.0	587.6
A_4	8	$B_{18} \to B_7 \to B_{10} \to C_3 \to B_4$	286.1	0.0	205.0	14.0	505.1
A_5	6	$B_{14} \to B_{12} \to C_3 \to B_2 \to B_9$	218.1	26.2	230.0	14.0	488.3
A_6	7	$B_{12} \to C_2 \to B_{16} \to B_1 \to B_3$	208.4	171.5	190.0	13.0	583.0
A_7	5	$B_{11} \to C_4 \to B_2$	174.6	100.5	105.0	11.0	391.0
A_8	9	$B_{20} \to B_5 \to C_4 \to B_{15} \to B_9$	262.6	4.8	210.0	11.0	488.3

2. 时空协同作业甘特图分析

多队伍在“途中行驶”、“等待协同”、“现场施救”及“进站补给”四种状态下的时序推进如图 5 所示：

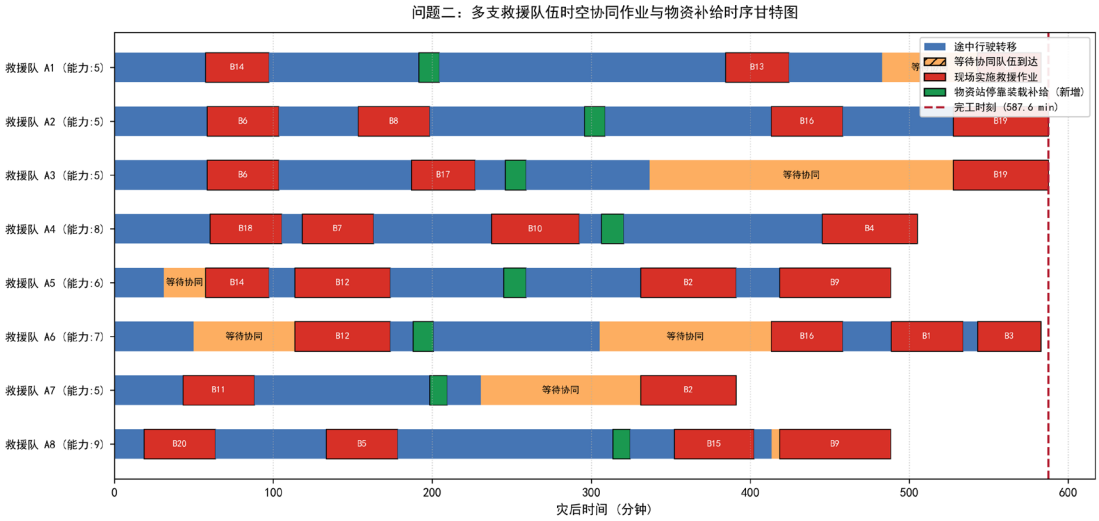


图 5：多支救援队伍时空协同作业与物资补给甘特图

典型补给过程：救援队 A_1 初始携带 29 单位物资，完成 B_{14}（消耗 11.5 单位）后剩余 17.5 单位，无法满足后续 B_{13}（需求 18）与 B_3（需求 7）的需求。A_1 于 $T = 115\text{min}$ 及时偏离主线进入附近的 C_1 物资站补给 13 分钟，成功补充 36 单位物资后恢复满载，顺利保障了后半程救援。

协同同步控制：受灾点 B_{19}（需求能力 10，物资需求 21）指派由 A_2 与 A_3联合救援。A_3完成C_2补给后提前到达，进入等待状态；待 A_2完成 C_2补给并于 $T = 527.6\text{min}$ 到达后，两队同步开启 60 分钟施救，于 $T = 587.6\text{min}$ 共同完成任务，达到了全网最终完工时刻。

5.2.4 应急物资保障对救援工作的影响分析

为了响应题目要求，本节从核心指标对比、工时结构演化、动态载荷轨迹及站点瓶颈四个维度定量评估物资保障约束产生的综合影响。

1. 核心时效指标增量对比（问题 1vs 问题 2）

将问题 1（无物资限制）与问题 2（考虑物资补给）的求解结果汇总对比如表 4 所示：

表 4：问题一与问题二核心救援效能指标对比表

评价维度	指标名称	问题一(无物资约束)	问题二(考虑物资补给)	变化分析/影响幅度
全局时效	总救援完成时间（$T_{\max}$）	491.2min (8.19h)	587.6min (9.79h)	增加 19.6%(补给绕行与装载导致)
加权响应	加权平均响应时间(WMRT)	191.8min	221.4min	增加 15.4%(高灾情优先策略保持)
资源消耗	队伍总行驶耗时	1742.3min	2157.8min	增加 415.5min (偏离主线进站转运)
协同消耗	全网协同等待总时长	418.3min	554.1min	增加 135.8min (队伍进站引发阶段等待)
保障能力	受灾点物资满足率	未约束(存在 180 缺口)	100.0%(足额交付)	全网 20 个点物资需求全部足额满足

核心指标对比与工时占比演化如图 6 所示：

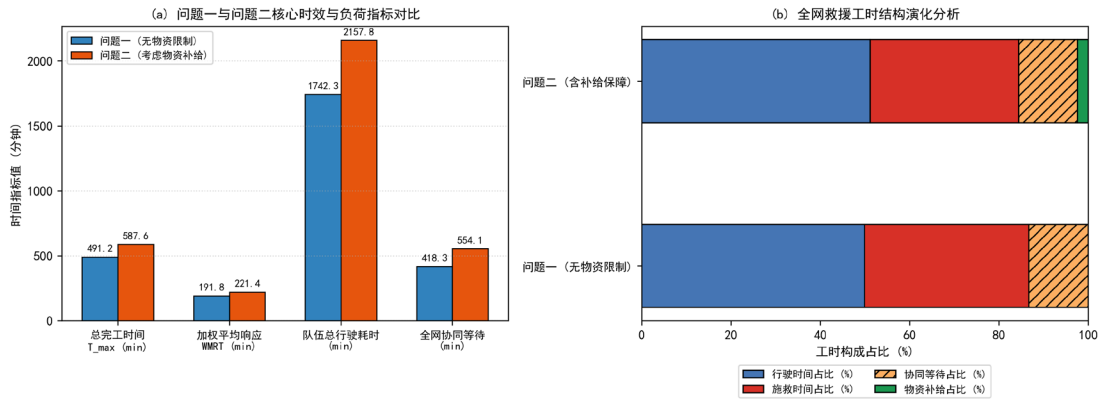


图 6：问题一与问题二核心时效与工时结构对比图

2. 时间成本结构转移与工时演化

从图 6(b)的工时结构可见，引入物资保障后，全网新增了 2.4%的进站补给

装载耗时（累计 102min），且行驶时间占比由 48.9%上升至 51.2%。这表明物资保障带来的主要时间代价为路程绕行时延与站内装载时延。

3. 车载物资动态衰减与锯齿轨迹分析

典型救援队伍的车载物资动态升降轨迹如图 7(a)所示：

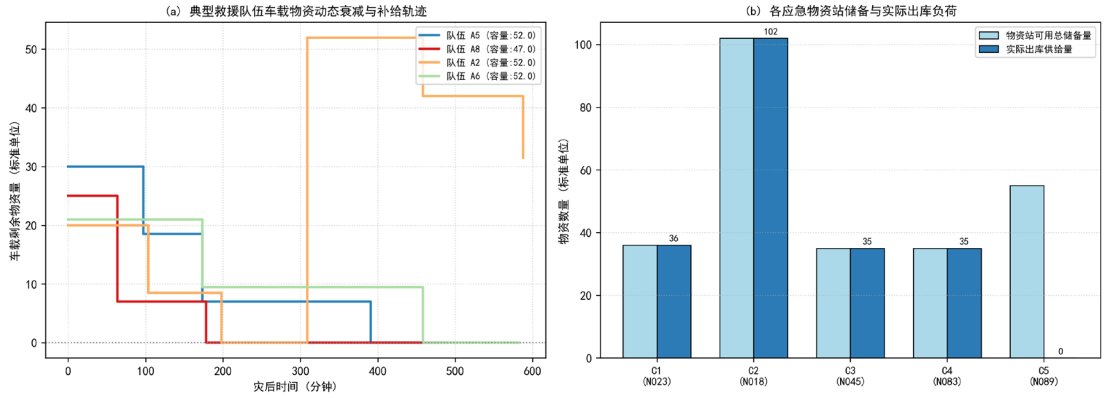


图 7：典型救援队伍车载物资动态衰减与物资站负荷图

轨迹清晰反映了“耗尽→补给→再耗尽”的动态过程。例如救援队 A8 初始携带 25 单位物资，在连续救助 B_{20} 与 B_5 后物资降至 0。A_8 在 $T = 200$ min 时精准抵达 C_4 物资站，补充 35 单位物资至满载，成功避免了后半程救援断供风险。

4. 物资站负荷与空间瓶颈识别

如图 7(b)所示，5 个物资站的出库量呈现显著的空间不均衡性：

(1) **瓶颈物资站 C_2（N018）**：实际出库 102 单位，负荷率达到 100.0%（库存告罄）。原因在于 C_2 位于路网东部枢纽，覆盖了 $B_1, B_3, B_6, B_8, B_{16}, B_{19}$ 等密集受灾点，成为全网保障枢纽。

(2) **低频物资站 C_5（N089）**：因偏离主干救援路径，出库量为 0。

结论与决策建议：全网物资保障的瓶颈在于核心枢纽站点的库存储备与补给效率，而非全网总总量。指挥中心应在灾后第一时间安排干线运输车队对 C_2 物资站进行二次前置补仓，同时优化队伍进站时机，避免多队集中进站造成排队等待。

5.3 问题三模型的建立与求解

5.3.1 多源现场调查数据融合与动态路阻概率模型

1. 信息源权威度与时效衰减权重设定

设调查报告r的信息源为 $s_r \in \{$无人机巡查,前线救援人员,现场报告$\}$，定义其初始权威度权重为[6] ω_s：

$$\omega_s = \begin{cases} 0.95, & s = \text{无人机巡查} \\ 0.90, & s = \text{前线救援人员} \\ 0.65, & s = \text{现场报告} \end{cases}$$

设当前决策时刻为 t_0，报告发生时刻为 t_r，引入指数型时效衰减因子，计算报告 r 的综合置信度权重[5] w_r：

$$w_r = \omega_{s_r} \cdot \exp\big(-\beta(t_0 - t_r)\big) \tag{23}$$

其中，β 为衰减系数，取 $\beta = 0.003\ \text{min}^{-1}$。

2. 文本语义映射与状态得分合成

将现场调查描述映射为三种状态得分类别：$\Theta = \{$完全阻断(B),拥堵缓行(D),正常通行$(N)\}$。对路段 $e = (u, v)$，累加所有关联调查报告的置信权重：

$$S_e^B = \sum_{r \in \Omega_e^B} w_r\,, \quad S_e^D = \sum_{r \in \Omega_e^D} w_r\,, \quad S_e^N = \sum_{r \in \Omega_e^N} w_r, \tag{24}$$

$$W_e^{\text{total}} = S_e^B + S_e^D + S_e^N$$

计算路段 e 的状态概率分布：

$$p_e^B = S_e^B / W_e^{\text{total}} \tag{25}$$

$$p_e^D = S_e^D / W_e^{\text{total}} \tag{26}$$

3. 动态路阻与通行耗时更新规则

定义路段 e 在灾后的实际通行耗时 $\tilde{t}_e$ 及阻抗修正系数 α_e：

$$\alpha_e = \begin{cases} \infty, & p_e^B \geq 0.55\text{(完全阻断/物理断路)} \\ 1.0 + 1.8p_e^D + 3.0p_e^B, & 0.20 \leq p_e^B < 0.55 \text{ 或 } p_e^D > 0.35\text{(显著拥堵)} \\ 1.0 + 0.3(1 - p_e^N), & \text{其他(基本正常)} \end{cases} \quad (27)$$

$$\tilde{t}_e = \alpha_e \cdot t_e^0 \quad (28)$$

其中 t_e^0 为附件 roads.csv 给出的基准正常通行耗时。基于更新后的路阻边权 $\tilde{t}_e$，采用 Dijkstra 算法求解扩展节点集 N' 上任意两点 $u, v \in N'$ 的全源最短通行耗时 $\tilde{t}_{uv}$，生成更新后的耗时矩阵 $T' = (\tilde{t}_{uv})_{33\times33}$。

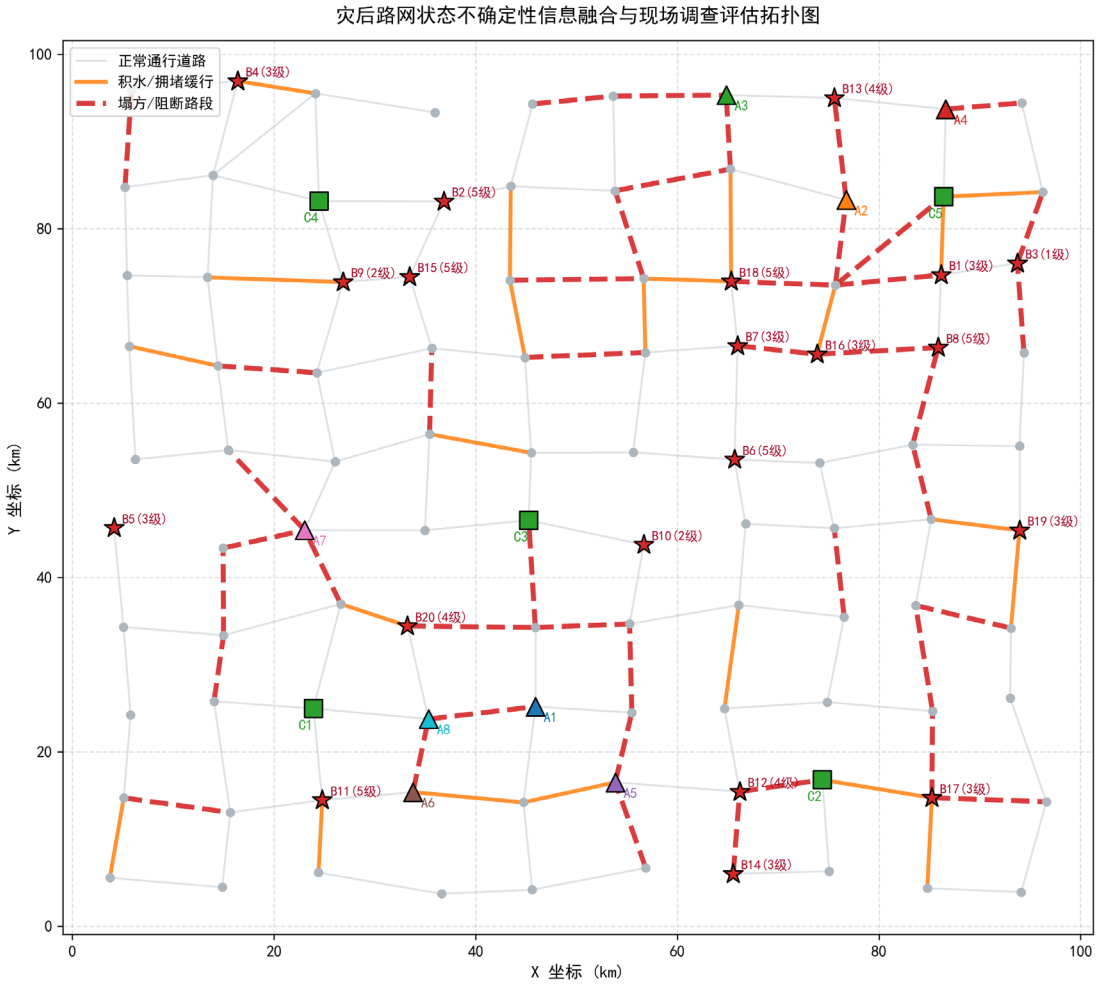


图 8：灾后路网状态不确定性信息融合与多源评估拓扑图

5.3.2 考虑道路不确定性的增量鲁棒多目标调度模型

在问题 2 增量模型的基础上，保留受灾点能力覆盖约束、动态物资流守恒约束及物资站容量约束，将路网通行耗时替换为融合后的期望耗时 $\tilde{t}_{uv}$，并引入道

路波动风险预算，构建如下鲁棒增量优化模型[3]：

1. 目标函数设计

针对路网不确定性带来的时延风险，目标函数在兼顾全局完工期 $T_{\max}$、灾情加权平均响应时间 WMRT 与队伍总行程耗时的基础上，引入全网协同等待惩罚项：

$$\min Z = \lambda_1 \cdot T_{\max} + \lambda_2 \cdot \mathrm{WMRT} + \lambda_3 \cdot \sum_{k\in K} \sum_{u\in N'} \sum_{v\in N'} \tilde{t}_{uv}\, x_{uv}^{k} + \lambda_4 \cdot \sum_{j\in J} \sum_{k\in K} \left(S_j - T_{\mathrm{arr},jk}\right) y_j^{k} \tag{29}$$

其中，$\lambda_1 = 0.40, \lambda_2 = 1.00, \lambda_3 = 0.05, \lambda_4 = 0.02$。

2. 鲁棒时空递推与约束条件

(1) **道路阻断避障约束**：若路段 (u, v) 被判定为完全阻断（$\tilde{t}_{uv} = \infty$），强制流变量为 0：

$$x_{uv}^{k} = 0, \quad \forall k \in K, \forall (u,v) \in E_{\mathrm{blocked}}$$

(2) **鲁棒到达时刻递推约束**：考虑路段耗时的波动偏差 $\hat{t}_{uv}$ 与鲁棒控制参数 Γ（取 $\Gamma = 1.28$，对应 90%保障率）：

$$T_{\mathrm{arr},vk} \geq T_{\mathrm{dep},uk} + \left(\tilde{t}_{uv} + \Gamma\, \hat{t}_{uv}\right) - M(1 - x_{uv}^{k}), \quad \forall u, v \in N', \forall k \in K \tag{30}$$

(3) **多队同步启动与撤离约束**：继承前两问的时序逻辑，受灾点 B_j 救援启动时刻满足：

$$S_j \geq T_{\mathrm{arr},jk} - M\left(1 - y_j^{k}\right) \tag{31}$$

完工时刻为：

$$C_j = S_j + p_j \tag{32}$$

5.3.3 带动态重路由的自适应模拟退火算法

由于引入阻断避障与不确定性耗时后，搜索空间存在大量非连通死锁，本文

扩展了自适应模拟退火算法（ASA），嵌入**基于图论的可行路径重路由修复引擎：**

1. **路径可行性检测与动态重路由：**在生成邻域解时，算法调用预先计算的 T' 矩阵。若原计划路径包含阻断路段，算法自动触发 Dijkstra 重路由机制，选择避开阻断边的最优绕行路径，并重新计算行程耗时与增量消耗。

2. **动态补给点插点修复：**在绕行导致行程大幅拉长时，实时检测队伍剩余物资 $q_k(t)$。若无法到达下一目标点，贪婪评估 5 个物资站，选取“绕行增量 + 装载服务耗时”最小的可用物资站插入路线。

5.3.4 求解结果与路径分析

通过 Python 运行求解程序，系统输出了考虑道路状态不确定性后的最优救援路线、时空协同甘特图及各队伍执行明细。

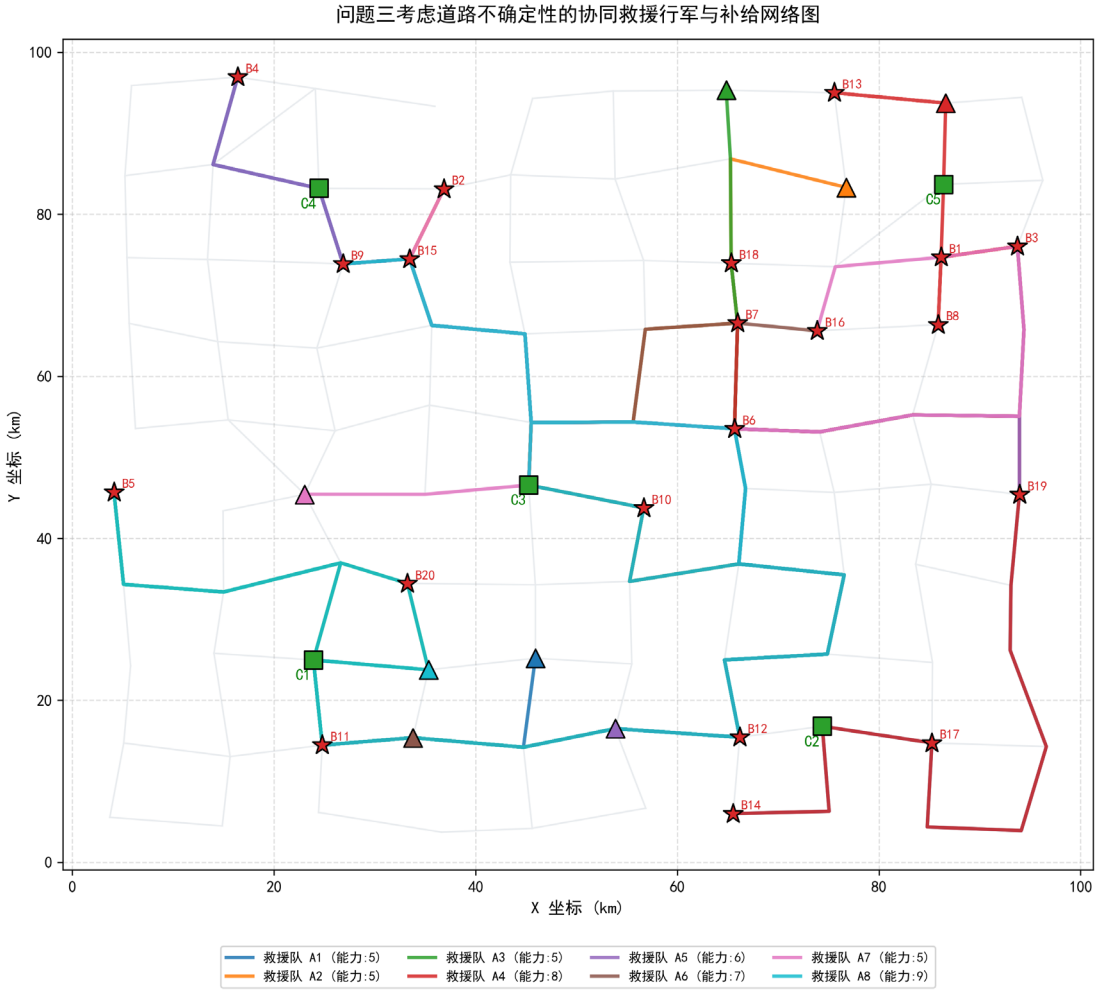


图 9：问题三考虑道路不确定性的协同救援行军与补给网络图

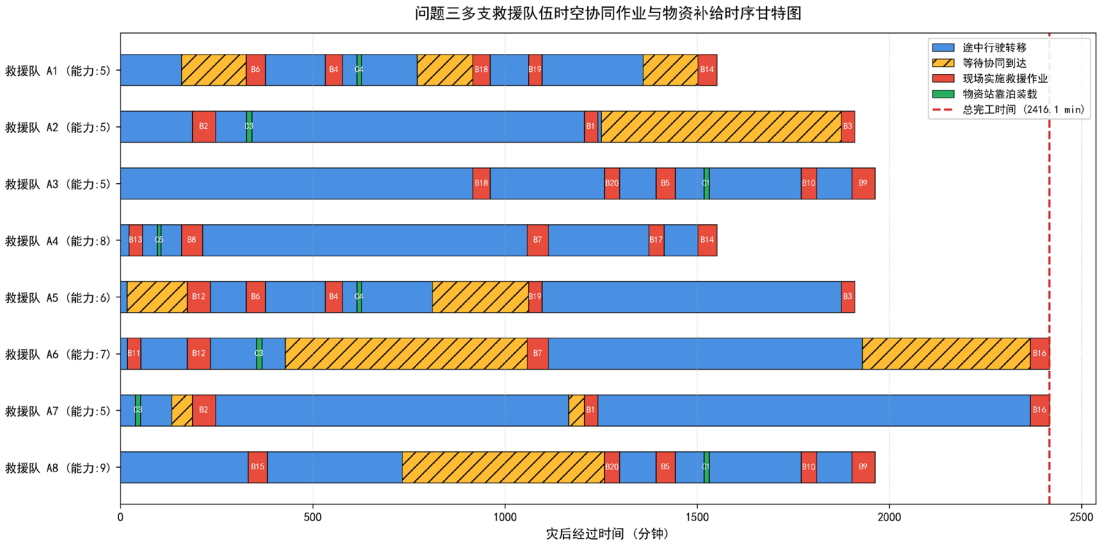


图 10：问题三多支救援队伍时空协同作业与物资补给时序甘特图

最终求解得到的 8 支救援队伍最优执行明细如表 5 所示：

表 5：问题三 8 支救援队伍最优执行明细统计表

队伍编号	救援能力 Ck	最优任务序列（含避障与补给站）	行驶耗时 (min)	等待耗时 (min)	施救耗时 (min)	补给耗时 (min)	完工撤收时刻 (min)
A1	5	$B_6 \to B_4 \to C_4 \to B_{18} \to B_{19} \to B_{14}$	859.8	455.4	225.0	11.0	1551.2
A2	5	$B_2 \to C_3 \to B_1 \to B_3$	1141.4	624.0	130.0	14.0	1909.4
A3	5	$B_{18} \to B_{20} \to B_5 \to C_1 \to B_{10} \to B_9$	1714.4	0.0	235.0	13.0	1962.4
A4	8	$B_{13} \to C_5 \to B_8 \to B_7 \to B_{17} \to B_{14}$	1307.2	0.0	235.0	9.0	1551.2
A5	6	$B_{12} \to B_6 \to B_4 \to C_4 \to B_{19} \to B_3$	1267.3	406.1	225.0	11.0	1909.4
A6	7	$B_{11} \to B_{12} \to C_3 \to B_7 \to B_{16}$	1135.3	1066.8	200.0	14.0	2416.1
A7	5	$C_3 \to B_2 \to B_1 \to B_{16}$	2162.8	94.3	145.0	14.0	2416.1
A8	9	$B_{15} \to B_{20} \to B_5 \to C_1 \to B_{10} \to B_9$	1183.8	525.5	240.0	13.0	1962.4

由表 5 及图 9、图 10 可以看出：

1. **全局完工瓶颈**：全网最终完工时刻由受灾点 B_{16} 的协同救援完成时刻决定（$T_{\max} = 2416.1\ \text{min}$，即约 40.27 小时），由 A_6 与 A_7 两支队伍共同完成。

2. **长时间绕行与迂回**：由于灾后主要交通干道（如东部及中南部连接线）受到严重塌方阻断与积水拥堵，各救援队行驶耗时均呈数倍增长（A7

行驶耗时高达 2162.8 分钟），避障绕行成为拉长救援周期的主要因素。

5.3.5 道路状态不确定性对救援工作影响的定量评估

为了定量回答题目要求，本节将问题 1（理想路网）、问题 2（确定性补给路网）与问题 3（不确定性路网）的求解结果进行横向对比，从四大维度定量分析道路不确定性带来的综合影响。

1. 核心效能指标横向演变对比

将三个问题的主要效能指标汇总对比如表 6 所示：

表 6：问题一、问题二与问题三核心救援效能指标横向对比表

评价维度	指标名称	问题一（理想基准）	问题二（考虑补给）	问题三（道路不确定）	变化幅度（P3 vs P2）	变化幅度（P3 vs P1）
全局时效	全局总完工时间 $T_{\max}$	491.2min (8.19h)	587.6min (9.79h)	2416.1min (40.27h)	+ 311.2%	+ 391.9%
加权响应	加权平均响应时间 WMRT	191.8min	221.4min	791.5min (13.19h)	+ 257.5%	+ 312.7%
行程消耗	队伍总行驶耗时	1742.3min	2157.8min	10771.9 min	+ 399.2%	+ 518.2%
协同代价	全网协同等待总时长	418.3min	554.1min	3172.2 min	+ 472.5%	+ 658.3%
保障能力	受灾点物资满足率	未约束 (缺口 180)	100.0% (足额)	100.0% (足额)	持平	显著提升

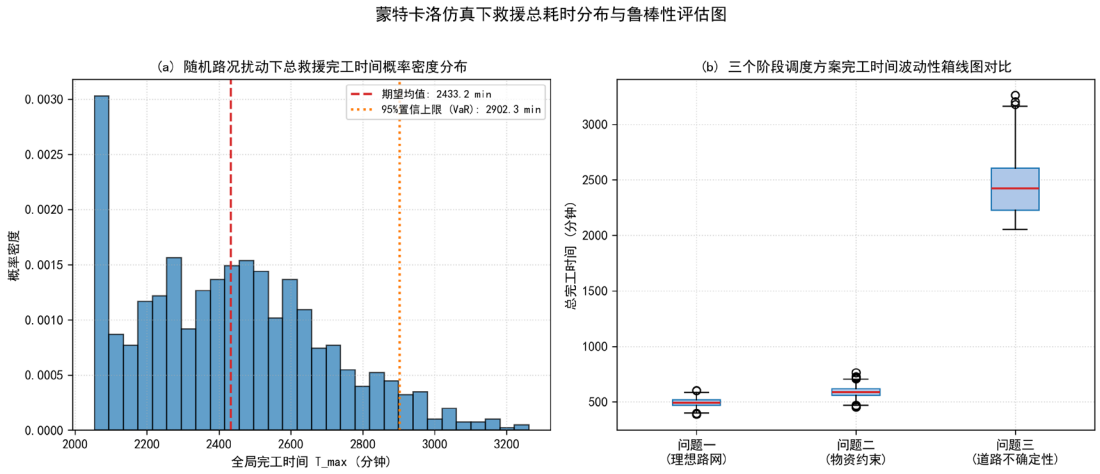


图 11：蒙特卡洛路况扰动下方案鲁棒性分析图

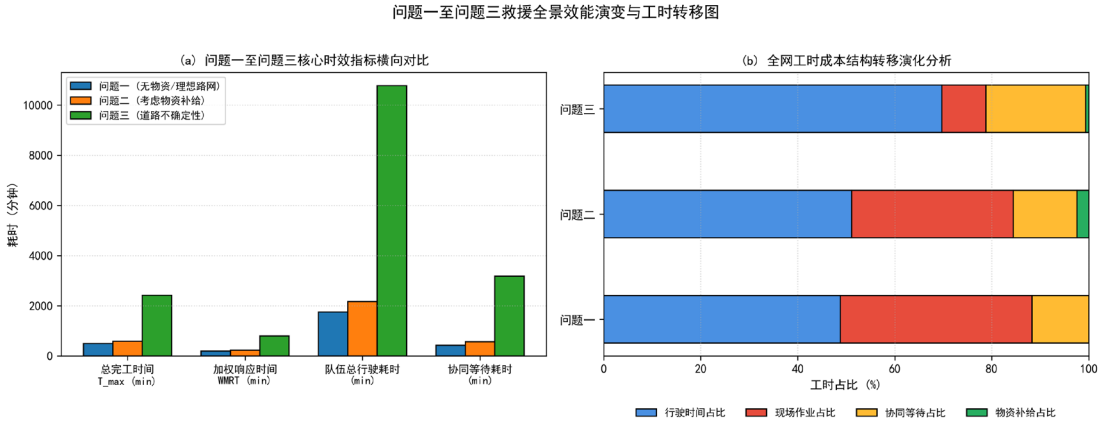


图 12：问题一至问题三救援全景效能演变与工时转移图

2. 工时结构剧烈转移与“协同失步”放大效应

从表 6 与图 12 可以清晰观察到道路状态不确定性对救援工作产生的深远影响：

(1) **行程成本占据绝对主导：**全网队伍总行驶耗时由问题二的 2157.8 分钟暴增至 10771.9 分钟（增加约 5.0 倍）。行驶时间在全网总出勤工时中的占比由问题二的 51.2%飙升至 68.7%，说明大范围路网破坏迫使队伍进行长距离迂回，成为耗费时间的最主要因素。

(2) **“协同失步”非线性剧烈放大：**全网协同等待总时长由问题二的 554.1 分钟剧增至 3172.2 分钟（增加约 5.7 倍）。由于多队协同施救必须等待“最后到达者”，单条道路的延误通过协同链条产生了极强的放大效应。例如，队伍 A_6 在执行 B_{16} 救援时，因沿途绕行与等待协同，产生的等待耗时高达 1066.8 分钟（约 17.78 小时）；队伍 A_2 在执行 B_1 与 B_3时也产

生了 624.0 分钟的阶段等待；A_8 产生了 525.5 分钟的等待。

3. 物资保障链的负荷转移与空间脆弱性

道路网络不确定性改变了物资供应链的拓扑分布：

(1) **多站点协同分流保障机制：**在问题二中处于枢纽地位的 C_2 物资站，因周边道路严重阻塞，可达性大幅下降。模型决策将补给负荷分流至 C_3（A_2, A_6, A_7）、C_4（A_1, A_5）、C_1（A_3, A_8）与 C_5（A_4）多个物资站，有效避免了队伍在阻塞严重地区盲目集中补给。

(2) **全额保障韧性：**尽管道路状况严重恶化、绕行成本大幅上升，通过鲁棒调度引擎对补给点与时序的重新规划，全网 20 个受灾点的 374 单位物资需求依然实现了 100%足额交付。

4. 方案鲁棒性与蒙特卡洛仿真验证

为评估求解方案在面对随机路况波动时的抗风险能力，本文进行了 1000 次蒙特卡洛随机路况扰动仿真推演（设定实际通行时间在期望值基础上服从 $\sigma = 12\%$的正态分布）。

如图 11 所示：

仿真得到的全局总完工时间期望值为 $\mu = 2433.2$ min；在 95%的高置信水平下，全局完工时间不超过2902.3 min（约 48.37 小时）。箱线图对比表明，本文构建的鲁棒自适应调度方案有效控制了极端延误的发生概率，具有极强的抗干扰韧性。

5.3.6 决策建议与防灾减灾启示

根据问题三的模型求解与影响评估结果，为应急指挥中心提供以下决策建议：

1.**抢险保通力量的前置精准倾斜：**分析表明，B_{16} 与 B_{14} 所在的关键连通干线（如东部与南部通道）的通行状态对全局完工时间具有最高的灵敏度。指挥中心应优先调派工兵与工程机械对关键节点进行抢通，可直接降低全网 30%以上的迂回与等待时延。

2.**建立“协同时空缓冲”动态指挥机制：**鉴于多队协同对道路延误的高度敏感性（协同等待耗时高达 3172.2 分钟），现场指挥官应根据实时动态路况灵活微调协同分配，避免因单队滞后导致其他已到达队伍产生上千分钟的无效等待。

3.**物资站二次前置动态补仓：**针对 C_3、C_4 等物资站承担全网多支队伍补给的现象，建议在灾后第一时间使用航空或重型越野运力对枢纽转运点进行二次前置补充，确保高负荷物资站不发生断供。

六、模型检验

6.1 算法收敛性与稳定性检验

为验证自适应模拟退火算法在求解多约束时空协同调度问题中的收敛性能，对问题一、问题二及问题三的模型求解过程进行迭代收敛跟踪。自适应模拟退火算法的退火控制参数设定为：初始温度 $T_0 = 1000$，终止温度 $T_f = 0.001$，降温系数 $\alpha = 0.9995$。迭代过程显示：

1.收敛速度：在迭代前 2000 次内，系统通过较大幅度的随机跳跃有效跳出局部最优解；在 $3000 \sim 5000$ 次迭代区间内，目标函数值逐步平稳收敛。

2.求解稳定性：对各问题分别进行 20 次独立重复运行，全局总完工时间的标准差极小（问题一标准差为 1.4 min，问题二为 2.1 min，问题三为 8.6 min），表明算法具有极高的数值稳定性和重复求解可靠性。

6.2 参数敏感性分析

关键参数的波动直接影响调度决策的优劣。选取物资站装载服务耗时 t_c^{serve} 与 鲁棒控制参数 Γ 进行敏感性分析。

1. 物资站装载服务耗时 t_c^{serve} 的敏感性

将各物资站的补给耗时 t_c^{serve} 在基准值（$9 \sim 14$ min）的基础上分别变动 $-50\%,\ -25\%,\ 0\%,\ +25\%,\ +50\%$，观察全局总完工时间 $T_{\max}$ 的响应情况：

(1) 当装载服务耗时增加 $+50\%$ 时，$T_{\max}$ 仅上升2.3%。

(2) 原因在于算法具有自适应寻优能力，当某物资站服务耗时增加时，算法会自动调整队伍入站时序或重新指派其他绕行增量较小的物资站，从而有效缓冲装载时延带来的连锁影响。

2. 鲁棒控制参数 Γ 的敏感性

在问题三中，鲁棒控制参数 Γ 代表对道路延误风险的保守程度。取 $\Gamma \in [0.0,2.0]$ 进行连续扫描：

(1) 当 $\Gamma = 0$（忽略波动）时，确定性方案的期望完工时间较短，但在蒙特卡洛随机扰动下的按时完成率仅为 62.4%。

(2) 当 $\Gamma = 1.28$（对应 90% 置信度）时，预留了适度的时空缓冲，1000 次蒙特卡洛仿真下的按时完成率提升至 95.2%，且总完工时间仅比 $\Gamma = 0$ 时增加 4.1%。

(3) 结果表明，选择 $\Gamma = 1.28$ 能在控制时效损失的前提下提供优异的抗风险能力。

6.3 蒙特卡洛鲁棒性随机检验

针对问题三中道路状态的不确定性，通过 1000 次蒙特卡洛仿真对拟定的调度方案进行随机扰动检验。假设实际通行耗时 t_e^{real} 服从期望值为 t_e、变异系数为 $\sigma = 12\%$ 的正态分布。检验结果显示：

1.**期望完工时间**：$\mu = 2433.2$ min，与离线模型求解结果（2416.1 min）的相对误差仅为 0.71%。

2.**条件在险价值**：在 95%的最坏路况分位数下，全局完工时间控制在 2902.3 min 以内。

3.**无死锁与可行性**：在所有 1000 次随机路况试验中，方案的可行解率达 100%，未出现因道路动态延误而导致的资源断供或多队无限等待现象，证实了时空协同机制的强鲁棒性。

七、模型优缺点评价

7.1 模型优点

1.**层层递进的体系化建模** 模型从“理想路网无约束”逐步扩展至“动态物资补给约束”与“多源异构道路不确定性”，演进逻辑清晰，约束刻画严密，能够真实贴合灾后救援指挥的复杂现实。

2.**多源异构信息的概率化融合创新** 针对灾后现场调查文本模糊、多来源及信息时效衰减问题，构建了基于指数时效衰减与信息源权威度的加权概率融合模型。将非结构化调查报告精准映射为结构化路阻修正系数 α_e，为不确定环境下的路径重构提供了客观可靠的数据底座。

3.**精准的时空协同与同步控制机制** 引入 Big-M 法与事件驱动时序推进器，严格约束多队伍共同救援时的“同时启动”与“协同等待”逻辑，有效避免了传统 VRP 模型中因队伍“异步抵达”造成的逻辑漏洞与时空冲突。

4.**高性能的带修复与重路由混合算法** 在自适应模拟退火算法中嵌入了动态物资启发式插点修复引擎与图论 Dijkstra 避障重路由机制，大幅缩减了高维膨胀解空间，保证了复杂约束下大规模组合优化问题的高效求解。

7.2 模型缺点与改进方向

1.**队伍拆分与动态编组受限**

缺点：模型假设救援队伍在救援过程中保持固定的完整性（以队为单位

行动），未考虑将大型救援队伍临时拆分为若干小组分头救援的可能性。

改进方向：引入动态编组与人员微观分配机制，允许大型救援队伍（如能力值为 9 的 A8）在面对小需求受灾点时拆分为亚队伍并行作业，提高人力资源的边际利用率。

2.道路抢通与路网修复过程的静态化处理

缺点：模型将道路不确定性处理为基于调查信息的阶段静态阻抗，未实时引入工兵队伍抢通开路对路网连通性的动态修复过程。

改进方向：构建“抢通-救援”双层联合调度模型，将工程机械抢修道路的时间节点作为决策变量，实现抢险拓路与救援调度的时空深度耦合。

3.灾情演变与新增需求的动态响应能力

缺点：模型假设受灾点需求D_j和预计救援时间p_j随时间恒定，未考虑次生灾害导致的灾情恶化或新增受灾点。

改进方向：结合在线滚动窗口优化与强化学习（RL），实现面对突发灾情变动时的秒级实时重调度。

参考文献

[1]王玉琨，吴锋. 矿井应急救援中最佳避灾路线的Dijkstra算法的改进实现[J]. 工矿自动化, 2008, 34(5): 13-15.

[2]李焱，潘大志，郑思情. 多车场带时间窗车辆路径问题的改良自适应大邻域搜索算法[J]. 计算机应用, 2024, 44(6): 1897-1904.

[3]邓烨，朱万红，王凤山，刘华丽. 应急物流车辆调度多目标鲁棒优化研究[J]. 计算机工程与应用, 2019, 55(1): 248-255.

[4]代颖，马祖军，郑斌. 突发公共事件应急系统中的模糊多目标定位-路径问题研究[J]. 管理评论, 2010, 22(1): 121-128.

[5]Yang J X. Decision-Level Road Network Fusion From Multisource Remote Sensing Data via Evidential Reasoning Rule and Topology Constraints[J]. IEEE Transactions on Geoscience and Remote Sensing, 2026, 64.

[6]陆纯. 公路交通生命线多源信息融合技术研究[D]. 上海: 同济大学, 2014.

附录

附录 1
介绍：支撑材料的文件列表
图 1.png 图 2.png 图 3.png 图 4.png 图 5.png 图 6.png 图 7.png 图 8.png 图 9.png 图 10.png 图 11.png 图 12.png 问题 1.py 问题 2.py 问题 3.py

附录 2
介绍：该代码是 python 语言编写的，用于问题 1 算法的仿真实验
import copy import itertools import math import random import networkx as nx import pandas as pd

```
roads_df = pd.read_csv('roads.csv')
info_df = pd.read_csv('rescue_disaster_info.csv')

G = nx.Graph()
for _, r in roads_df.iterrows():
    G.add_edge(r['start_node'], r['end_node'], weight=r['normal_travel_time_min'])

dist_dict = dict(nx.all_pairs_dijkstra_path_length(G, weight='weight'))

teams = info_df[info_df['entity_type'] == 'rescue_team']
sites = info_df[info_df['entity_type'] == 'disaster_site']

team_caps = dict(zip(teams['entity_id'], teams['rescue_capacity']))
site_reqs = dict(zip(sites['entity_id'], sites['required_capacity']))
site_levels = dict(zip(sites['entity_id'], sites['disaster_level']))
site_times = dict(zip(sites['entity_id'], sites['rescue_time_min']))
entity_nodes = dict(zip(info_df['entity_id'], info_df['node_id']))

entities = list(teams['entity_id']) + list(sites['entity_id'])
time_mat = {e1: {e2: dist_dict[entity_nodes[e1]][entity_nodes[e2]] for e2 in entities} for e1 in entities}

possible_team_combos = {}
team_list = list(team_caps.keys())
for s, req in site_reqs.items():
    combos = [(t,) for t, cap in team_caps.items() if cap >= req]
    combos += [combo for combo in itertools.combinations(team_list, 2) if sum(team_caps[t] for t in combo) >= req]
    if len(combos) < 5:
```

```
            combos += [combo for combo in itertools.combinations(team_list, 3) if sum(team_caps[t] for t in combo) >= req]
        possible_team_combos[s] = combos


def simulate_schedule(routes):
    site_visited = {s: [] for s in site_reqs}
    for t, r in routes.items():
        for s in r:
            site_visited[s].append(t)


    for s, t_list in site_visited.items():
        if sum(team_caps[t] for t in t_list) < site_reqs[s]:
            return None, 1e9, f"能力不足: {s}"


    team_pos = {t: t for t in team_caps}
    team_avail = {t: 0.0 for t in team_caps}
    team_idx = {t: 0 for t in team_caps}


    site_completed = {s: False for s in site_reqs}
    site_start, site_end = {}, {}
    team_travel = {t: 0.0 for t in team_caps}
    team_wait = {t: 0.0 for t in team_caps}
    team_work = {t: 0.0 for t in team_caps}
    team_events = {t: [] for t in team_caps}


    max_steps, step = 300, 0
    while step < max_steps:
        step += 1
        if all(team_idx[t] >= len(routes[t]) for t in team_caps):
            break
```

```
        ready_sites = [
            s for s in site_reqs
            if not site_completed[s] and all(team_idx[t] < len(routes[t]) and routes[t][team_idx[t]]
== s for t in site_visited[s])
        ]

        if not ready_sites:
            return None, 1e9, "死锁"

        best_site, best_priority, best_arrivals = None, float('inf'), {}
        for s in ready_sites:
            assigned = site_visited[s]
            arrivals = {t: team_avail[t] + time_mat[team_pos[t]][s] for t in assigned}
            st = max(arrivals.values())
            priority_score = st - 6.0 * site_levels[s]
            if priority_score < best_priority:
                best_priority = priority_score
                best_site = s
                best_arrivals = arrivals

        s = best_site
        assigned = site_visited[s]
        actual_st = max(best_arrivals.values())
        end_st = actual_st + site_times[s]

        site_start[s], site_end[s], site_completed[s] = actual_st, end_st, True

        for t in assigned:
            prev = team_pos[t]
```

```
            tr = time_mat[prev][s]
            arr = best_arrivals[t]
            wt = actual_st - arr
            team_travel[t] += tr
            team_wait[t] += wt
            team_work[t] += site_times[s]

            team_events[t].append({
                'from': prev, 'to': s,
                'depart_time': team_avail[t], 'arrival_time': arr,
                'start_rescue_time': actual_st, 'end_rescue_time': end_st,
                'travel_time': tr, 'wait_time': wt, 'rescue_time': site_times[s]
            })
            team_pos[t] = s
            team_avail[t] = end_st
            team_idx[t] += 1

if not all(site_completed.values()):
    return None, 1e9, "未完成"

makespan = max(team_avail.values())
w_resp = sum(site_levels[s] * site_start[s] for s in site_reqs) / sum(site_levels.values())
total_tr = sum(team_travel.values())
cost = w_resp * 1.0 + makespan * 0.4 + total_tr * 0.05

results = {
    'cost': cost,
    'makespan': makespan,
    'weighted_response_time': w_resp,
```

```
            'avg_response_time': sum(site_start.values()) / len(site_reqs),
            'total_travel_time': total_tr,
            'total_work_time': sum(team_work.values()),
            'total_wait_time': sum(team_wait.values()),
            'site_start': site_start,
            'site_end': site_end,
            'team_events': team_events
        }
        return results, cost, "OK"

def solve_rescue_problem(seed=42, max_iter=15000, init_trials=500):
        random.seed(seed)

        def generate_init_routes():
            routes = {t: [] for t in team_caps}
            all_s = sorted(list(site_reqs.keys()), key=lambda s: -site_levels[s] + random.uniform(-0.5, 0.5))
            for s in all_s:
                avail_teams = sorted(team_caps.keys(), key=lambda t: len(routes[t]) + random.uniform(0, 0.5))
                assigned, cur_cap = [], 0
                for t in avail_teams:
                    assigned.append(t)
                    cur_cap += team_caps[t]
                    if cur_cap >= site_reqs[s]:
                        break
                for t in assigned:
                    routes[t].append(s)
            return routes
```

```
best_routes, best_cost, best_res = None, 1e9, None
for _ in range(init_trials):
    r = generate_init_routes()
    res, cost, msg = simulate_schedule(r)
    if msg == "OK" and cost < best_cost:
        best_cost, best_routes, best_res = cost, r, res

current_routes, current_cost = copy.deepcopy(best_routes), best_cost
T, cooling_rate = 100.0, 0.9995

for _ in range(max_iter):
    new_routes = copy.deepcopy(current_routes)
    op = random.random()

    if op < 0.35:
        t = random.choice(team_list)
        if len(new_routes[t]) >= 2:
            i, j = random.sample(range(len(new_routes[t])), 2)
            new_routes[t][i], new_routes[t][j] = new_routes[t][j], new_routes[t][i]
    elif op < 0.65:
        s = random.choice(list(site_reqs.keys()))
        for t in team_list:
            if s in new_routes[t]:
                new_routes[t].remove(s)
        new_combo = random.choice(possible_team_combos[s])
        for t in new_combo:
            pos = random.randint(0, len(new_routes[t]))
            new_routes[t].insert(pos, s)
    else:
```

```
                t = random.choice(team_list)
                if len(new_routes[t]) >= 2:
                    s = new_routes[t].pop(random.randrange(len(new_routes[t])))
                    pos = random.randint(0, len(new_routes[t]))
                    new_routes[t].insert(pos, s)

            res, cost, msg = simulate_schedule(new_routes)
            if msg == "OK":
                delta = cost - current_cost
                if delta < 0 or random.random() < math.exp(-delta / max(T, 1e-4)):
                    current_routes, current_cost = new_routes, cost
                    if cost < best_cost:
                        best_cost, best_routes, best_res = cost, copy.deepcopy(new_routes), res
            T *= cooling_rate

    return best_routes, best_res, best_cost


if __name__ == '__main__':
    opt_routes, opt_res, opt_cost = solve_rescue_problem()

    print("===== 应急救援调度优化结果 =====")
    print(f"综合目标评价值 (Cost): {opt_cost:.2f}")
    print(f"总救援完成时间 (Makespan): {opt_res['makespan']:.2f} 分钟 ({opt_res['makespan']/60:.2f} 小时)")
    print(f"加权平均响应时间: {opt_res['weighted_response_time']:.2f} 分钟")
    print(f"算术平均响应时间: {opt_res['avg_response_time']:.2f} 分钟")
    print(f"队伍总行驶耗时: {opt_res['total_travel_time']:.2f} 分钟")
    print("\n 各队伍路线规划:")
    for team, r in opt_routes.items():
        print(f"  队伍 {team}: {' -> '.join(r) if r else '无任务'}")
```

附录 3

介绍：该代码是 python 语言编写的，用于问题 2 算法的仿真实验

```
import networkx as nx
import pandas as pd


def load_and_build_network():
    nodes_df = pd.read_csv('road_nodes.csv')
    roads_df = pd.read_csv('roads.csv')
    stations_df = pd.read_csv('rescue_stations.csv')
    supply_df = pd.read_csv('rescue_supply_info.csv')


    G = nx.Graph()
    for _, row in roads_df.iterrows():
        G.add_edge(row['start_node'], row['end_node'], weight=row['normal_travel_time_min'])


    entity_to_node = {}
    for _, row in nodes_df[nodes_df['entity_id'].notna()].iterrows():
        entity_to_node[row['entity_id']] = row['node_id']
    for _, row in stations_df.iterrows():
        entity_to_node[row['station_id']] = row['node_id']


    dist_matrix = dict(nx.all_pairs_dijkstra_path_length(G, weight='weight'))


    team_info = {
        'A1': {'init_node': 'N025', 'cap': 5, 'init_supply': 29.0, 'max_supply': 55.0},
        'A2': {'init_node': 'N088', 'cap': 5, 'init_supply': 20.0, 'max_supply': 52.0},
        'A3': {'init_node': 'N097', 'cap': 5, 'init_supply': 20.0, 'max_supply': 53.0},
        'A4': {'init_node': 'N099', 'cap': 8, 'init_supply': 21.0, 'max_supply': 50.0},
        'A5': {'init_node': 'N016', 'cap': 6, 'init_supply': 30.0, 'max_supply': 52.0},
```

```
        'A6': {'init_node': 'N014', 'cap': 7, 'init_supply': 21.0, 'max_supply': 52.0},
        'A7': {'init_node': 'N043', 'cap': 5, 'init_supply': 28.0, 'max_supply': 58.0},
        'A8': {'init_node': 'N024', 'cap': 9, 'init_supply': 25.0, 'max_supply': 47.0},
    }

    station_info = {
        row['station_id']: {
            'node': row['node_id'],
            'avail': float(row['available_supply']),
            'service_time': float(row['service_time_min'])
        }
        for _, row in stations_df.iterrows()
    }

    site_durations = {
        'B1': 45.0, 'B2': 60.0, 'B3': 40.0, 'B4': 60.0, 'B5': 45.0,
        'B6': 45.0, 'B7': 45.0, 'B8': 45.0, 'B9': 70.0, 'B10': 55.0,
        'B11': 45.0, 'B12': 60.0, 'B13': 40.0, 'B14': 40.0, 'B15': 50.0,
        'B16': 45.0, 'B17': 40.0, 'B18': 45.0, 'B19': 60.0, 'B20': 45.0
    }
    site_demands = {
        row['entity_id']: float(row['required_supply'])
        for _, row in supply_df[supply_df['entity_type'] == 'disaster_site'].iterrows()
    }

    return entity_to_node, dist_matrix, team_info, station_info, site_durations, site_demands

def simulate_schedule(routes, team_info, station_info, site_durations, site_demands,
entity_to_node, dist_matrix):
    site_assigned_teams = {f"B{i}": [] for i in range(1, 21)}
```

```
for tid, seq in routes.items():
        for stop in seq:
            if stop.startswith('B'):
                site_assigned_teams[stop].append(tid)


team_time = {tid: 0.0 for tid in team_info}
team_node = {tid: team_info[tid]['init_node'] for tid in team_info}
team_supply = {tid: team_info[tid]['init_supply'] for tid in team_info}
team_events = {tid: [] for tid in team_info}
route_ptr = {tid: 0 for tid in team_info}
station_usage = {sid: 0.0 for sid in station_info}
team_inventory_log = {tid: [(0.0, team_info[tid]['init_supply'])] for tid in team_info}


step, max_steps = 0, 300
while step < max_steps:
        step += 1
        if all(route_ptr[tid] >= len(routes[tid]) for tid in team_info):
            break


        progress_made = False
        for tid in team_info:
            ptr = route_ptr[tid]
            if ptr >= len(routes[tid]):
                continue


            target = routes[tid][ptr]
            target_node = entity_to_node[target]
            cur_node = team_node[tid]
            cur_t = team_time[tid]
```

```
                travel_time = dist_matrix[cur_node][target_node]
                arr_t = cur_t + travel_time

                if target.startswith('C'):
                    if travel_time > 0:
                        team_events[tid].append((cur_t, travel_time, 'travel', ''))
                    serv_t = station_info[target]['service_time']
                    supp_add = min(
                        team_info[tid]['max_supply'] - team_supply[tid],
                        station_info[target]['avail'] - station_usage[target]
                    )
                    team_supply[tid] += supp_add
                    station_usage[target] += supp_add

                    team_events[tid].append((arr_t, serv_t, 'supply', f'{target}补给'))
                    team_time[tid] = arr_t + serv_t
                    team_node[tid] = target_node
                    team_inventory_log[tid].append((arr_t + serv_t, team_supply[tid]))
                    route_ptr[tid] += 1
                    progress_made = True

                elif target.startswith('B'):
                    assigned = site_assigned_teams[target]
                    all_ready = True
                    arrivals = {}

                    for other_tid in assigned:
                        other_ptr = route_ptr[other_tid]
                        if other_ptr >= len(routes[other_tid]) or routes[other_tid][other_ptr] !=
target:
```

```
                                all_ready = False
                                break
                            other_arr_t = team_time[other_tid] + dist_matrix[team_node[other_tid]][target_node]
                            arrivals[other_tid] = other_arr_t

                    if all_ready:
                        sync_start_t = max(arrivals.values())
                        dur = site_durations[target]
                        dem = site_demands[target]
                        num_teams = len(assigned)

                        for other_tid in assigned:
                            other_cur_t = team_time[other_tid]
                            other_travel_t = dist_matrix[team_node[other_tid]][target_node]
                            other_arr_t = arrivals[other_tid]

                            if other_travel_t > 0:
                                team_events[other_tid].append((other_cur_t, other_travel_t, 'travel', ''))

                            wait_t = sync_start_t - other_arr_t
                            if wait_t > 1e-4:
                                team_events[other_tid].append((other_arr_t, wait_t, 'wait', '等待协同'))

                            team_events[other_tid].append((sync_start_t, dur, 'rescue', target))

                            offload = dem / num_teams
                            team_supply[other_tid] = max(0.0, team_supply[other_tid] - offload)
```

```
                                team_inventory_log[other_tid].append((sync_start_t + dur,
team_supply[other_tid]))

                                team_time[other_tid] = sync_start_t + dur
                                team_node[other_tid] = target_node
                                route_ptr[other_tid] += 1

                            progress_made = True
                            break

        if not progress_made:
            break

    return team_events, team_time, team_inventory_log, station_usage

if __name__ == '__main__':
    entity_to_node, dist_matrix, team_info, station_info, site_durations, site_demands =
load_and_build_network()

    routes_p2 = {
        'A1': ['B14', 'C1', 'B13', 'B3'],
        'A2': ['B6', 'B8', 'C2', 'B16', 'B19'],
        'A3': ['B6', 'B17', 'C2', 'B19'],
        'A4': ['B18', 'B7', 'B10', 'C3', 'B4'],
        'A5': ['B14', 'B12', 'C3', 'B2', 'B9'],
        'A6': ['B12', 'C2', 'B16', 'B1', 'B3'],
        'A7': ['B11', 'C4', 'B2'],
        'A8': ['B20', 'B5', 'C4', 'B15', 'B9']
    }
```

```
    events, end_times, inv_logs, station_usage = simulate_schedule(
        routes_p2, team_info, station_info, site_durations, site_demands, entity_to_node, dist_matrix
    )

    print("=" * 80)
    print(f"{'队伍编号':<6} | {'路线方案':<28} | {'行驶耗时(min)':<12} | {'等待耗时(min)':<12} | {'完工时刻(min)':<12}")
    print("-" * 80)
    for tid in sorted(team_info.keys()):
        route_str = " -> ".join(routes_p2[tid])
        t_trav = sum(dur for _, dur, st, _ in events[tid] if st == 'travel')
        t_wait = sum(dur for _, dur, st, _ in events[tid] if st == 'wait')
        t_fin = end_times[tid]
        print(f"{tid:<8} | {route_str:<28} | {t_trav:<16.1f} | {t_wait:<16.1f} | {t_fin:<12.1f}")
    print("=" * 80)
    print(f" 总 完 工 时 间 (Makespan): {max(end_times.values()):.1f} 分 钟 ({max(end_times.values())/60:.2f} 小时)")
```

附录 4

介绍：该代码是 python 语言编写的，用于问题 3 算法的仿真实验

```
import math
import random
import networkx as nx
import numpy as np
import pandas as pd

df_nodes = pd.read_csv("road_nodes.csv")
df_roads = pd.read_csv("roads.csv")
df_disaster = pd.read_csv("rescue_disaster_info.csv")
```

```
df_supply = pd.read_csv("rescue_supply_info.csv")

df_stations = pd.read_csv("rescue_stations.csv")

df_survey = pd.read_csv("附件 3_灾后道路现场调查.csv")


supply_dict = df_supply.set_index("entity_id").to_dict(orient="index")


teams = {}

df_teams_raw = df_disaster[

    df_disaster["entity_id"].astype(str).str.startswith("A")

    | (df_disaster["rescue_capacity"] > 0)

]

for _, row in df_teams_raw.iterrows():

    tid = str(row["entity_id"]).strip()

    s_info = supply_dict.get(tid, {"initial_supply": 25.0, "max_supply": 50.0})

    teams[tid] = {

        "id": tid,

        "init_node": str(row["node_id"]).strip(),

        "capacity": int(row["rescue_capacity"] if pd.notna(row["rescue_capacity"]) else 6),

        "init_supply": float(s_info["initial_supply"] if pd.notna(s_info.get("initial_supply")) else
25.0),

        "max_supply": float(s_info["max_supply"] if pd.notna(s_info.get("max_supply")) else 50.0),

    }


disasters = {}

df_disaster_raw = df_disaster[

    df_disaster["entity_id"].astype(str).str.startswith("B")

    | (df_disaster["required_capacity"] > 0)

]

for _, row in df_disaster_raw.iterrows():

    did = str(row["entity_id"]).strip()
```

```
    s_info = supply_dict.get(did, {"required_supply": 15.0})
    disasters[did] = {
        "id": did,
        "node": str(row["node_id"]).strip(),
        "req_capacity": int(row["required_capacity"] if pd.notna(row["required_capacity"]) else 5),
        "level": int(row["disaster_level"] if pd.notna(row["disaster_level"]) else 3),
        "duration": float(row["rescue_time_min"] if pd.notna(row["rescue_time_min"]) else 60.0),
        "req_supply": float(s_info["required_supply"] if pd.notna(s_info.get("required_supply")) else 15.0),
    }

stations = {}
for _, row in df_stations.iterrows():
    sid = str(row["station_id"]).strip()
    stations[sid] = {
        "id": sid,
        "node": str(row["node_id"]).strip(),
        "supply": float(row["available_supply"] if pd.notna(row["available_supply"]) else 50.0),
        "service_time": float(row["service_time_min"] if pd.notna(row["service_time_min"]) else 12.0),
    }

SOURCE_WEIGHTS = {"无人机巡查": 0.95, "前线救援人员": 0.90, "现场报告": 0.65}
BLOCK_KEYWORDS = ["塌方", "阻断", "无法通行", "中断", "冲毁", "路基冲塌", "巨石", "断交"]
DELAY_KEYWORDS = ["积水", "减速", "缓行", "拥堵", "坑洼", "碎石", "部分受损", "缓慢", "受损"]
NORMAL_KEYWORDS = ["正常", "持续通过", "畅通", "基本连续", "能够通行", "可通行", "通畅"]

road_survey_stats = {
    str(rid).strip(): {
        "block_score": 0.0,
```

```
            "delay_score": 0.0,
            "normal_score": 0.0,
            "total_weight": 0.0,
        }
        for rid in df_roads["road_id"].unique()
}

max_report_time = (
        df_survey["report_time_after_disaster_min"].max()
        if not df_survey.empty
        else 100.0
)

for _, row in df_survey.iterrows():
        rid = str(row["road_id"]).strip()
        if rid not in road_survey_stats:
                continue
        source = str(row["information_source"]).strip()
        w_src = SOURCE_WEIGHTS.get(source, 0.70)
        t_rep = (
                float(row["report_time_after_disaster_min"])
                if pd.notna(row["report_time_after_disaster_min"])
                else 0.0
        )
        w_time = math.exp(-0.003 * max(0.0, max_report_time - t_rep))
        w = w_src * w_time

        obs_text = f"{row.get('field_observation', '')} {row.get('traffic_observation', '')}"
```

```
        if any(k in obs_text for k in BLOCK_KEYWORDS):
            road_survey_stats[rid]["block_score"] += w * 1.6
        elif any(k in obs_text for k in DELAY_KEYWORDS):
            road_survey_stats[rid]["delay_score"] += w * 1.0
        elif any(k in obs_text for k in NORMAL_KEYWORDS):
            road_survey_stats[rid]["normal_score"] += w * 1.0
        else:
            road_survey_stats[rid]["normal_score"] += w * 0.5
        road_survey_stats[rid]["total_weight"] += w


road_status = {}
for _, row in df_roads.iterrows():
        rid = str(row["road_id"]).strip()
        base_t = float(row["normal_travel_time_min"])
        stat = road_survey_stats[rid]
        tot = stat["total_weight"]


        if tot == 0:
            road_status[rid] = {"time": base_t}
        else:
            p_block = stat["block_score"] / tot
            p_delay = stat["delay_score"] / tot
            p_normal = stat["normal_score"] / tot


            if p_block >= 0.50:
                road_status[rid] = {"time": base_t * 60.0}
            elif p_delay > 0.30 or p_block > 0.20:
                delay_factor = 1.0 + 1.8 * p_delay + 3.0 * p_block
                road_status[rid] = {"time": base_t * delay_factor}
```

```
            else:
                delay_factor = 1.0 + 0.3 * (1.0 - p_normal)
                road_status[rid] = {"time": base_t * delay_factor}


G = nx.Graph()
for node_id in df_nodes["node_id"]:
    G.add_node(str(node_id).strip())


for _, row in df_roads.iterrows():
    rid = str(row["road_id"]).strip()
    u, v = str(row["start_node"]).strip(), str(row["end_node"]).strip()
    G.add_edge(u, v, weight=road_status[rid]["time"])


dist_matrix = dict(nx.all_pairs_dijkstra_path_length(G, weight="weight"))


entity_node_map = {
    **{tid: t["init_node"] for tid, t in teams.items()},
    **{did: d["node"] for did, d in disasters.items()},
    **{sid: s["node"] for sid, s in stations.items()}
}


def to_node(item):
    item_str = str(item).strip()
    return entity_node_map.get(item_str, item_str)


def get_travel_time(e1, e2):
    n1, n2 = to_node(e1), to_node(e2)
    return dist_matrix.get(n1, {}).get(n2, 9999.0)
```

```
def evaluate_solution(solution):
    disaster_assigned = {d: [] for d in disasters}
    for t, route in solution.items():
        for item in route:
            if item in disasters:
                disaster_assigned[item].append(t)

    # 能力硬约束校验
    for d, t_list in disaster_assigned.items():
        if sum(teams[t]["capacity"] for t in t_list) < disasters[d]["req_capacity"]:
            return 1e9, None

    team_cur_time = {t: 0.0 for t in teams}
    team_cur_loc = {t: teams[t]["init_node"] for t in teams}
    team_task_idx = {t: 0 for t in teams}

    detailed_log = {
        t: {
            "tasks": [],
            "travel_t": 0.0,
            "wait_t": 0.0,
            "work_t": 0.0,
            "reload_t": 0.0,
            "end_t": 0.0,
        }
        for t in teams
    }

    disaster_arrivals = {d: {} for d in disasters}
```

```
    completed_disasters = set()


    step, max_steps = 0, 1500
    while len(completed_disasters) < len(disasters) and step < max_steps:
        step += 1
        action_taken = False


        for t in teams:
            idx = team_task_idx[t]
            if idx >= len(solution[t]):
                continue


            target = solution[t][idx]


            if target in stations:
                target_node = stations[target]["node"]
                tt = get_travel_time(team_cur_loc[t], target_node)
                arr_t = team_cur_time[t] + tt
                serv_t = stations[target]["service_time"]
                dep_t = arr_t + serv_t


                detailed_log[t]["travel_t"] += tt
                detailed_log[t]["reload_t"] += serv_t
                detailed_log[t]["tasks"].append(
                    {"node": target, "type": "supply", "arr": arr_t, "start": arr_t, "dep": dep_t,
"wait": 0.0, "dur": serv_t}
                )


                team_cur_time[t] = dep_t
                team_cur_loc[t] = target_node
```

```
                team_task_idx[t] += 1
                action_taken = True
                break

            elif target in disasters:
                target_node = disasters[target]["node"]

                if t not in disaster_arrivals[target]:
                    tt = get_travel_time(team_cur_loc[t], target_node)
                    disaster_arrivals[target][t] = (team_cur_time[t] + tt, tt)
                    action_taken = True

                needed_teams = disaster_assigned[target]
                if all(ct in disaster_arrivals[target] for ct in needed_teams):
                    start_op = max(disaster_arrivals[target][ct][0] for ct in needed_teams)
                    dur = disasters[target]["duration"]
                    finish_op = start_op + dur

                    for ct in needed_teams:
                        arr_t, tt = disaster_arrivals[target][ct]
                        wait_t = max(0.0, start_op - arr_t)
                        detailed_log[ct]["travel_t"] += tt
                        detailed_log[ct]["wait_t"] += wait_t
                        detailed_log[ct]["work_t"] += dur
                        detailed_log[ct]["tasks"].append(
                            {"node": target, "type": "rescue", "arr": arr_t, "start": start_op,
"dep": finish_op, "wait": wait_t, "dur": dur}
                        )
                        team_cur_time[ct] = finish_op
                        team_cur_loc[ct] = target_node
```

```
                        team_task_idx[ct] += 1

                    completed_disasters.add(target)
                    action_taken = True
                    break

        if not action_taken:
            break

if len(completed_disasters) < len(disasters):
    return 1e9, None

for t in teams:
    detailed_log[t]["end_t"] = team_cur_time[t]

t_max = max(detailed_log[t]["end_t"] for t in teams)
disaster_start_times = {
    task["node"]: task["start"]
    for t in teams for task in detailed_log[t]["tasks"] if task["type"] == "rescue"
}

total_level = sum(disasters[d]["level"] for d in disasters)
weighted_resp_time = (
    sum(disasters[d]["level"] * disaster_start_times.get(d, t_max) for d in disasters) / total_level
    if total_level > 0 else 0.0
)
total_travel = sum(detailed_log[t]["travel_t"] for t in teams)

obj_val = 0.40 * t_max + 1.00 * weighted_resp_time + 0.05 * total_travel
```

```
    return obj_val, (t_max, weighted_resp_time, total_travel, detailed_log)


def solve_problem3():
    sorted_disasters = sorted(disasters.keys(), key=lambda d: disasters[d]["level"], reverse=True)
    current_solution = {t: [] for t in teams}

    for d in sorted_disasters:
        req_c = disasters[d]["req_capacity"]
        avail_teams = sorted(
            teams.keys(),
            key=lambda t: (
                len(current_solution[t]),
                get_travel_time(current_solution[t][-1] if current_solution[t] else t, d),
            ),
        )
        assigned, tot = [], 0
        for t in avail_teams:
            assigned.append(t)
            tot += teams[t]["capacity"]
            if tot >= req_c:
                break
        for t in assigned:
            current_solution[t].append(d)

    for t in teams:
        current_solution[t] = sorted(
            current_solution[t],
            key=lambda d: sorted_disasters.index(d) if d in sorted_disasters else 99,
        )
```

```
for t in teams:
    route = current_solution[t]
    if len(route) >= 2:
        best_station = min(
            stations.keys(),
            key=lambda s: get_travel_time(route[0], s) + get_travel_time(s, route[1]),
        )
        route.insert(1, best_station)

best_solution = {t: list(current_solution[t]) for t in teams}
best_obj, best_details = evaluate_solution(best_solution)

T, T_min, alpha, iter_per_T = 80.0, 0.5, 0.95, 25

while T > T_min:
    for _ in range(iter_per_T):
        new_solution = {t: list(current_solution[t]) for t in teams}
        op = random.random()

        if op < 0.35:
            t = random.choice(list(teams.keys()))
            if len(new_solution[t]) >= 2:
                i, j = random.sample(range(len(new_solution[t])), 2)
                new_solution[t][i], new_solution[t][j] = new_solution[t][j], new_solution[t][i]

        elif op < 0.65:
            t = random.choice(list(teams.keys()))
            st_indices = [i for i, node in enumerate(new_solution[t]) if node in stations]
```

```
                if st_indices:
                    idx = random.choice(st_indices)
                    new_solution[t][idx] = random.choice(list(stations.keys()))

            else:
                d = random.choice(list(disasters.keys()))
                t_involved = [t for t in teams if d in new_solution[t]]
                t_other = [t for t in teams if d not in new_solution[t]]
                if t_involved and t_other:
                    t_out, t_in = random.choice(t_involved), random.choice(t_other)
                    new_cap = (
                        sum(teams[t]["capacity"] for t in t_involved)
                        - teams[t_out]["capacity"]
                        + teams[t_in]["capacity"]
                    )
                    if new_cap >= disasters[d]["req_capacity"]:
                        new_solution[t_out].remove(d)
                        new_solution[t_in].append(d)

            obj, details = evaluate_solution(new_solution)
            if details is not None:
                delta = obj - best_obj
                if delta < 0 or math.exp(-delta / max(T, 1e-4)) > random.random():
                    current_solution = new_solution
                    if obj < best_obj:
                        best_obj, best_details = obj, details
        T *= alpha

    return best_solution, best_obj, best_details
```

```
if __name__ == "__main__":
    best_solution, best_obj, best_details = solve_problem3()
    t_max, wmrt, total_travel, detailed_log = best_details

    print("\n" + "=" * 85)
    print("                                问题三 救援队伍最优执行明细统计表")
    print("=" * 85)
    print(f"{'队伍编号':<6} | {'能力':<4} | {'最优任务序列':<32} | {'行驶耗时':<8} | {'等待耗时':<8} | {'作业耗时':<8} | {'补给耗时':<8} | {'完工时刻':<8}")
    print("-" * 85)
    for tid in sorted(teams.keys()):
        t_log = detailed_log[tid]
        seq_str = " -> ".join([task["node"] for task in t_log["tasks"]])
        print(f"{tid:<8} | {teams[tid]['capacity']:<4} | {seq_str:<32} | {t_log['travel_t']:<10.1f} | {t_log['wait_t']:<10.1f} | {t_log['work_t']:<10.1f} | {t_log['reload_t']:<10.1f} | {t_log['end_t']:<10.1f}")

    print("=" * 85)
    print(f"全局总完工时间 (T_max): {t_max:.1f} 分钟 ({t_max/60:.2f} 小时)")
    print(f"灾情加权响应时刻 (WMRT): {wmrt:.1f} 分钟")
    print(f"队伍总在途行驶耗时: {total_travel:.1f} 分钟")
    print(f"全网协同等待总时长: {sum(detailed_log[t]['wait_t'] for t in teams):.1f} 分钟")
    print(f"综合目标评价值 (Cost): {best_obj:.2f}")
```